

VERILOG · HOMEWORK

8-bit Adder 결과의 4-Digit FND 십진수(BCD) 표시 RTL 설계 · 합성 · 검증

Basys3 (Artix-7) | 조합 회로 → FSM 스캔 → 클럭 분주 3단계 개선

131,072 전수(Exhaustive, i_cin 0/1) 시뮬레이션 + 보드 실측



UPDATE 2026.10.06 Reset 개념 · Counter 스캔 · Clock Divider · Duty 분석

8-bit Sum (0~510) Shown on All 4 FND Digits at Once

목표

두 8bit 입력의 합 $i_a + i_b$ (0~510)를 BCD 4자리로 변환하여 Basys3 FND에 십진수로 표시하는 RTL 설계·합성

진행 경과

- | | |
|----|---|
| v1 | 조합 회로 + 버튼(i_{sel})으로 자리 선택 → 1자리씩만 표시 |
| v2 | FSM으로 Anode 자동 스캔 (100 MHz) → 고스팅(잔상) 발생 |
| v3 | 클럭 분주(1 kHz) 후 스캔 → 4자리 동시 정상 표시 |

검증 방법 평가 요약 — 전수 입력·자동 판정은 우수. 다만 Anode 순서·자리 갱신을 확인하지 않아 **AN3 미구동·BCD4 누락 결함**을 **PASS**로 통과시킴을 결함 주입으로 확인 → 제안 TB로 보완 (p.28~36)

131,072

전수 입력 조합 (i_{cin} 0/1)

[PASS] $err_cnt = 0$

3 단계

v1 → v2 → v3

문제 발견 → 원인 분리 → 개선

10_{ns} → 1_{ms}

자리당 점등 시간

(Basys3 권장 ≥ 0.25 ms 충족)

250 Hz

최종 화면 갱신 주기 (4 ms)

깜빡임·잔상 없음

최대값 $255 + 255 = 510$ 보드 실측 표시 완료

Carry(o_cout) LED 동시 점등

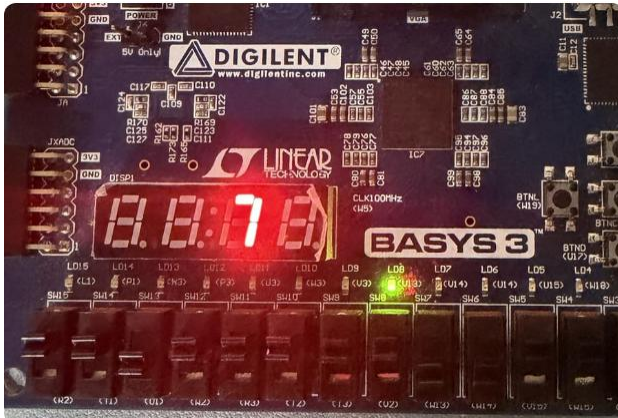
10/6 업데이트 — Reset 개념 (p.7~9) · Counter 스캔 · Clock Divider 원리 · Duty 분석 (p.37~42)

Root Cause Analysis and Fix

v1 조합 회로

top_adder_fnd_8bit

- adder → digit_splitter → **mux_4x1** → fnd_decoder
- **decoder_2x4**가 버튼 i_sel로 Anode 선택
- TB 65,536 전수 **PASS**

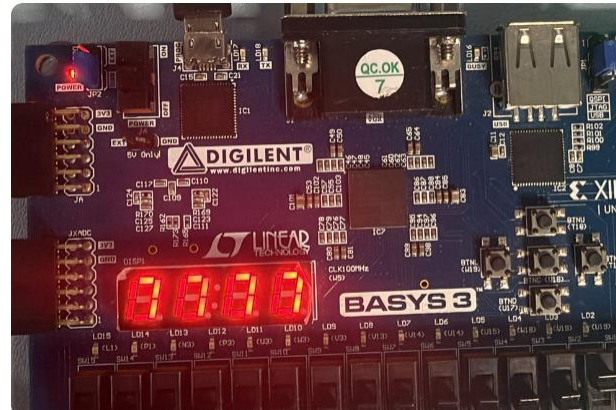


문제 — 버튼을 눌러야 한 자리씩만 보임.
합을 한눈에 읽을 수 없음

v2 FSM 자동 스캔

top_adder_fnd_for_clk (분주 전)

- mux + decoder → **Fnd_Controller**(FSM) 대체
- 100 MHz 클럭으로 AN0→AN3 순차 점등
- TB 131,072 전수 **PASS** (i_cin 0/1)

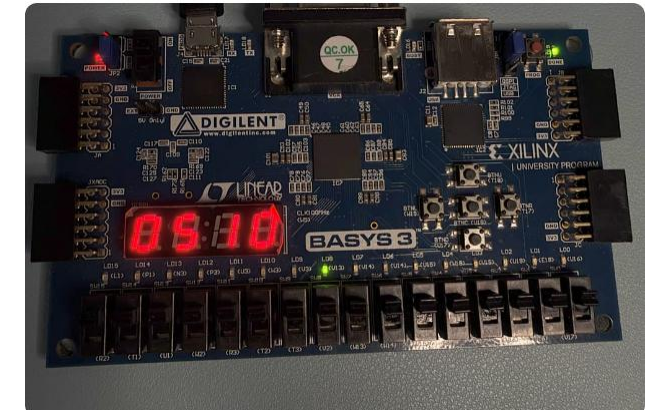


문제 — FSM 버그 수정 후에도 숫자가 옆
자리에 겹쳐 보임 (고스팅)

v3 클럭 분주 스캔

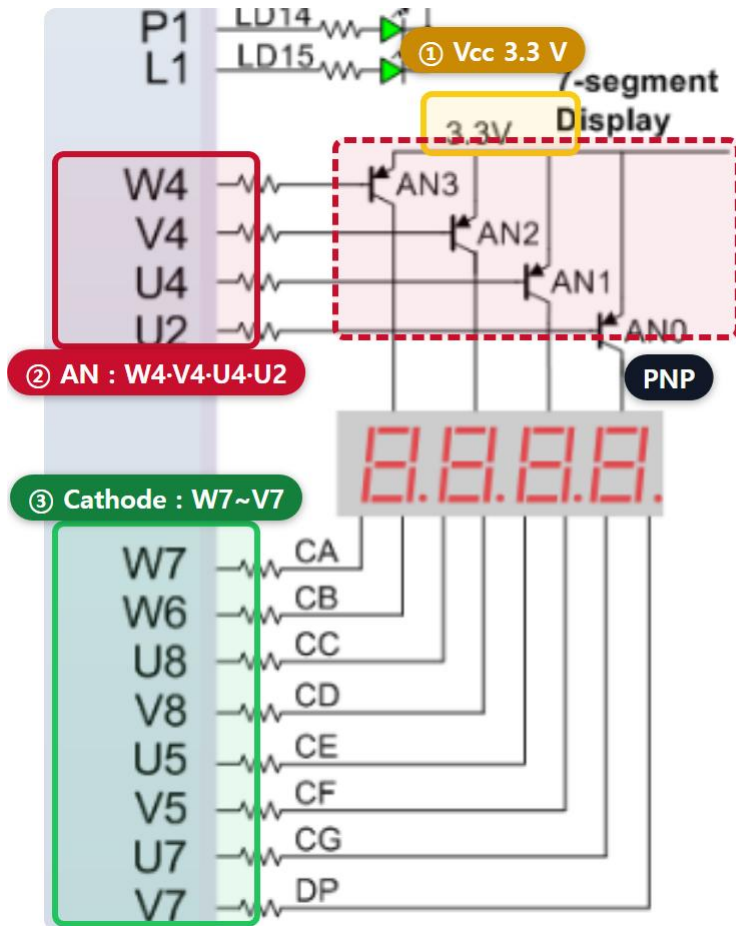
top_adder_fnd_for_clk + clock_div

- **clock_div** 추가: 100 MHz → 1 kHz
- 자리당 1 ms, 화면 갱신 4 ms (250 Hz)
- 보드 실측 정상 표시



결과 — 4자리 동시 표시, 잔상 없음. 최대
값 510 확인

Basys3 FND Is Common Anode — Anode and Cathode Both Active-Low



1 Emitter에 Vcc(3.3 V) 상시 인가

4자리의 Anode는 각각 PNP 트랜지스터를 통해 3.3 V에 연결

2 Base(AN) = 0 V → 해당 자리 ON

PNP는 Base가 Low일 때 ON → 따라서 o_fnd_com은 **Active-Low Signal**

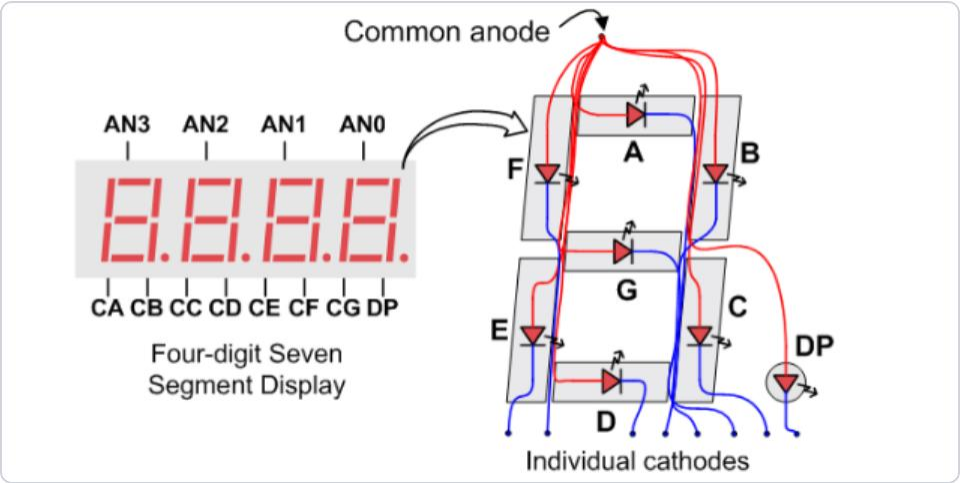
1110 = AN0(일의 자리)만 ON

3 Cathode = 0 → 해당 세그먼트 ON

8개 Cathode(CA~CG, DP)는 **4자리가 공유** → 한 순간에는 한 자리만 원하는 숫자 표시 가능

설계 시사점 — 4자리를 동시에 보이려면 Anode를 빠르게 돌아가며 켜는 **시분할 스캔**이 필수

Segment Assert Condition — Segment Lights When Cathode = 0

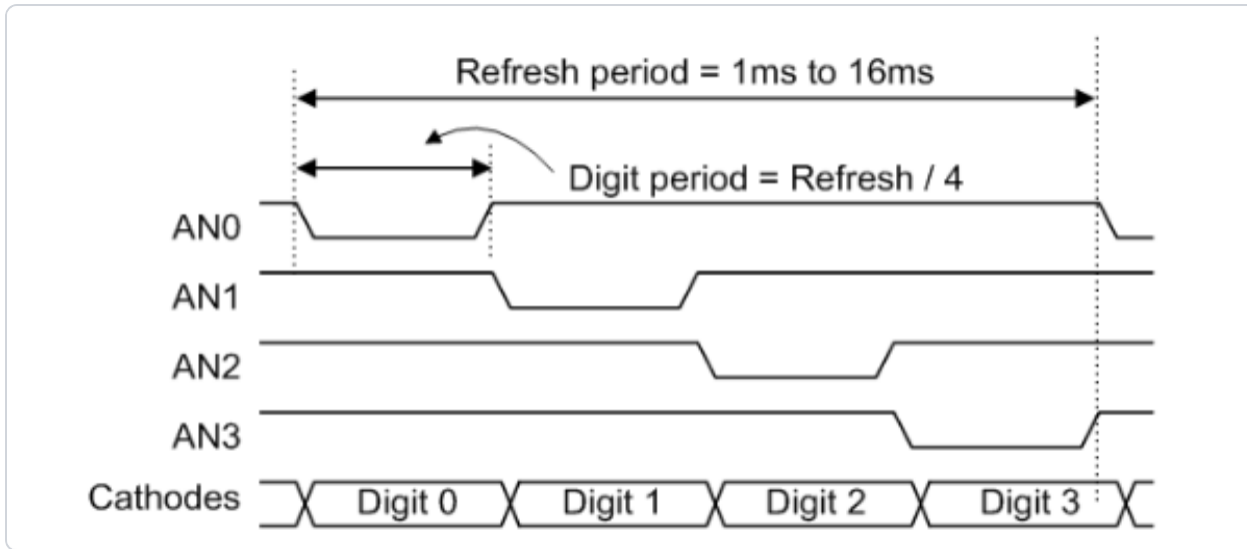


예) 숫자 '1' — B, C만 점등
DP G F E D C B A
1 1 1 1 1 0 0 1 = 8'hF9

숫자	표시	DP	G	F	E	D	C	B	A	HEX
0		1	1	0	0	0	0	0	0	C0
1		1	1	1	1	1	0	0	1	F9
2		1	0	1	0	0	1	0	0	A4
3		1	0	1	1	0	0	0	0	B0
4		1	0	0	1	1	0	0	1	99
5		1	0	0	1	0	0	1	0	92
6		1	0	0	0	0	0	1	0	82
7		1	1	1	1	1	0	0	0	F8
8		1	0	0	0	0	0	0	0	80
9		1	0	0	1	0	0	0	0	90

0 = 점등 (Active-Low) · o_fnd_data[7:0] = {DP, G, F, E, D, C, B, A} · fnd_decoder.v 기준

Time-Multiplexed Scan Timing — ≥ 0.25 ms per Digit



출처: Digilent Basys3 Reference Manual — 7-Segment Display timing

1 ~ 16 ms

Refresh period — 4자리를 한 바퀴 도는 시간
16 ms 초과 시 깜빡임(Flicker)

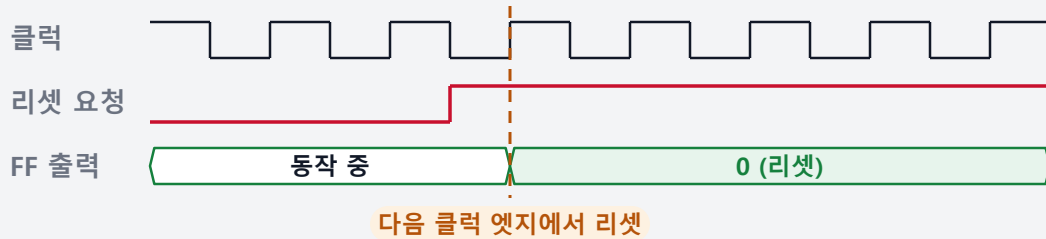
≥ 0.25 ms

Digit period = Refresh / 4
이보다 짧으면 트랜지스터·LED가 따라오지 못해 잔상(고스팅)

Anode 지연 — AN 전환 시 PNP 트랜지스터 On/Off 지연이 존재 →
자리당 점등 시간이 이를 충분히 덮어야 함

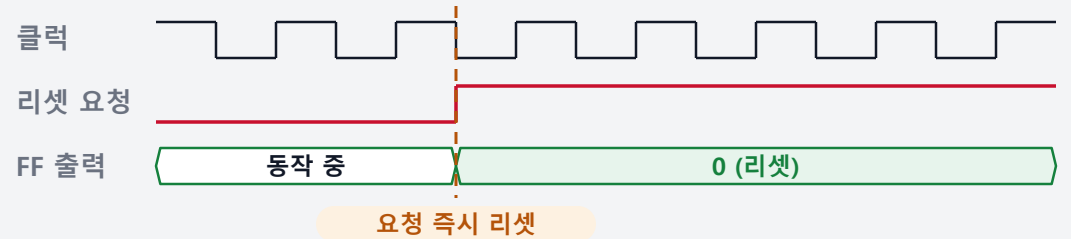
Sync vs Async Reset — Async Acts Instantly, Sync Waits for the Clock Edge

동기식 리셋 Synchronous — 클럭 엣지에서만 리셋



- ✓ 모든 FF가 같은 클럭 엣지에서 리셋·해제 → 로직 간 동기가 맞음
- ✓ 엣지 사이에 들어온 짧은 글리치는 무시
- ✓ 리셋도 일반 데이터처럼 타이밍 분석 가능
- ✗ 클럭이 없으면 리셋이 안 먹음 (클럭 정지, PLL 안정 전)
- ✗ 리셋을 최소 1클럭 이상 유지해야 함

비동기식 리셋 Asynchronous — 클럭과 무관하게 즉시 리셋



- ✓ 클럭 없이도 즉시 리셋 → 빠름, 전원 투입 직후에도 동작
- ✓ 데이터 경로에 리셋 로직이 끼지 않음
- ✗ 해제 순간이 클럭 엣지와 겹치면 FF가 불안정 (메타스테이빌리티)
- ✗ 멀리 있는 로직은 리셋 해제가 늦게 도착 → 일부 FF만 한 클럭 늦게 출발
- ✗ 리셋 선에 글리치가 생겨도 그대로 리셋

FPGA

설정(Configuration)이 끝나면 **클럭이 안정된 상태로 시작**하고 모든 FF가 초기값으로 자동 설정
→ **동기식(또는 꼭 필요한 FF만) 리셋 권장** — FF·메모리·DSP 레지스터가 동기 리셋 구조

칩 설계 (ASIC)

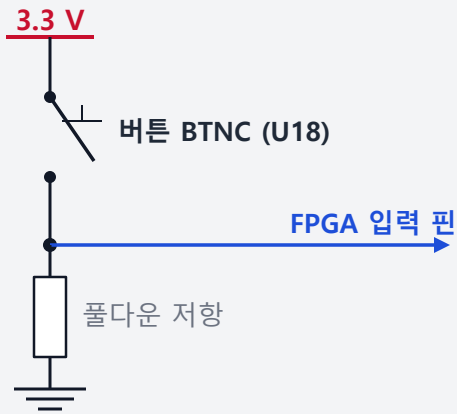
동기식 리셋을 많이 씀 — 타이밍 검증이 단순하고 글리치에 강함
단, 전원 투입 시 클럭이 없을 수 있어 '**걸 때는 비동기, 풀 때는 동기**' 조합이 일반적 (다음 장)

이 설계 (Bsys3)

분주기·FSM·카운터 모두 **비동기 리셋** — 버튼을 누르는 즉시 반영
버튼을 놓는 순간은 클럭에 맞추지 않음 → 지금은 문제 없지만 회로가 커지면 보완 필요

Active-High Button, Active-Low Logic — The Top Inverts the Reset Once

① Basys3 버튼 = Active-High

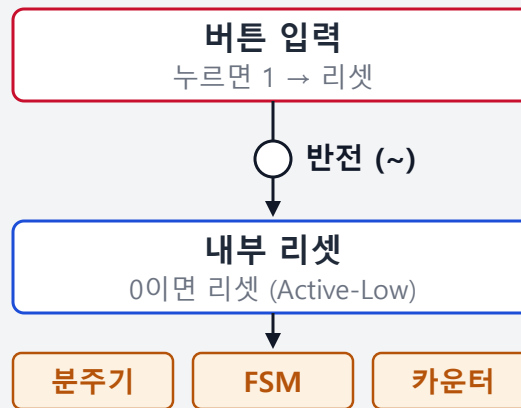


떼면 저항이 0 V로 당김 → 0

누르면 3.3 V가 연결 → 1 (Active-High)

개념도 — Basys3 버튼은 누를 때 High

② Top에서 한 번만 반전



Active-High로 받은 이유

보드 버튼(GPIO)이 Active-High라서 핀은 그대로 받고, 내부 규칙(Active-Low)에 맞게 Top에서만 반전

③ 현업에서 Active-Low를 쓰는 이유

1 전원 투입 순간

전원이 올라오는 동안 선은 0 V → 별도 회로 없이 '리셋 중'으로 시작

2 한 선을 여럿이 공유

풀업 + 오픈 드레인 → 어느 장치든 Low로 끌어내리면 전체 리셋

3 TTL 시절 관습

Low로 끌어내리는 쪽이 전류 구동에 유리 → RST_n, CS_n처럼 제어 핀이 Low-Active

4 칩 표준 셀

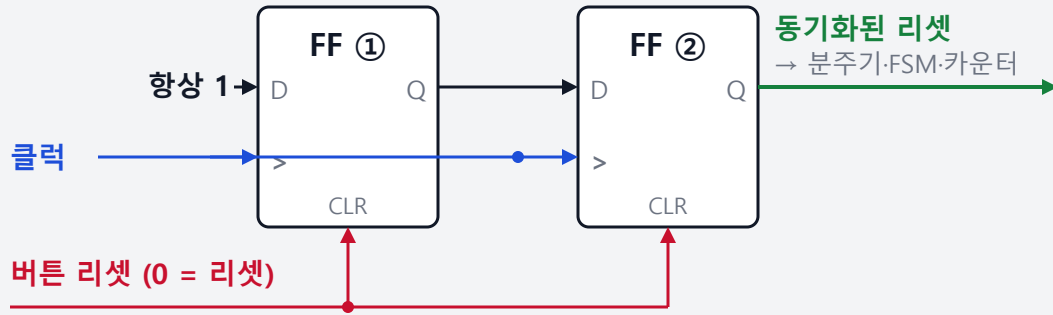
리셋 FF가 Low-Active 핀으로 제공 → 인버터 없이 바로 연결

FPGA 안에서는 반대 — Active-High가 더 자연스러움

- Xilinx 7-Series FF의 Set/Reset 핀은 **Active-High** → Active-High 리셋은 그대로 연결되고, Active-Low는 반전 로직이 한 단 더 필요할 수 있음 (Xilinx 권장: Active-High)
- 정리 — **칩 사이·보드 리셋 선은 Active-Low, FPGA 내부 로직은 Active-High**가 일반적
- 이 설계는 버튼(High) → Top 반전 → 내부 Low로 받아 **칩 설계 관습**을 따른 형태. FPGA만 쓴다면 내부도 Active-High로 두는 편이 자원상 유리

In Practice — Assert Reset Asynchronously, Release It Synchronously

리셋 동기화기 — FF 2개 (Reset Synchronizer)



동작 타이밍



버튼을 아무 때나 놓아도 **모든 FF가 같은 클럭 엣지에서 동시에 출발** → 동기·비동기 장점을 함께 얻는 현업 표준 구조

FF가 2개인 이유

첫 FF는 해제 순간이 클럭 엣지와 겹치면 잠깐 불안정할 수 있음 → **한 클럭 더 기다려 안정된 값만 출력** (메타스테이빌리티 차단)

두 방식의 장점만

걸 때는 클럭이 없어도 즉시 (비동기 장점)

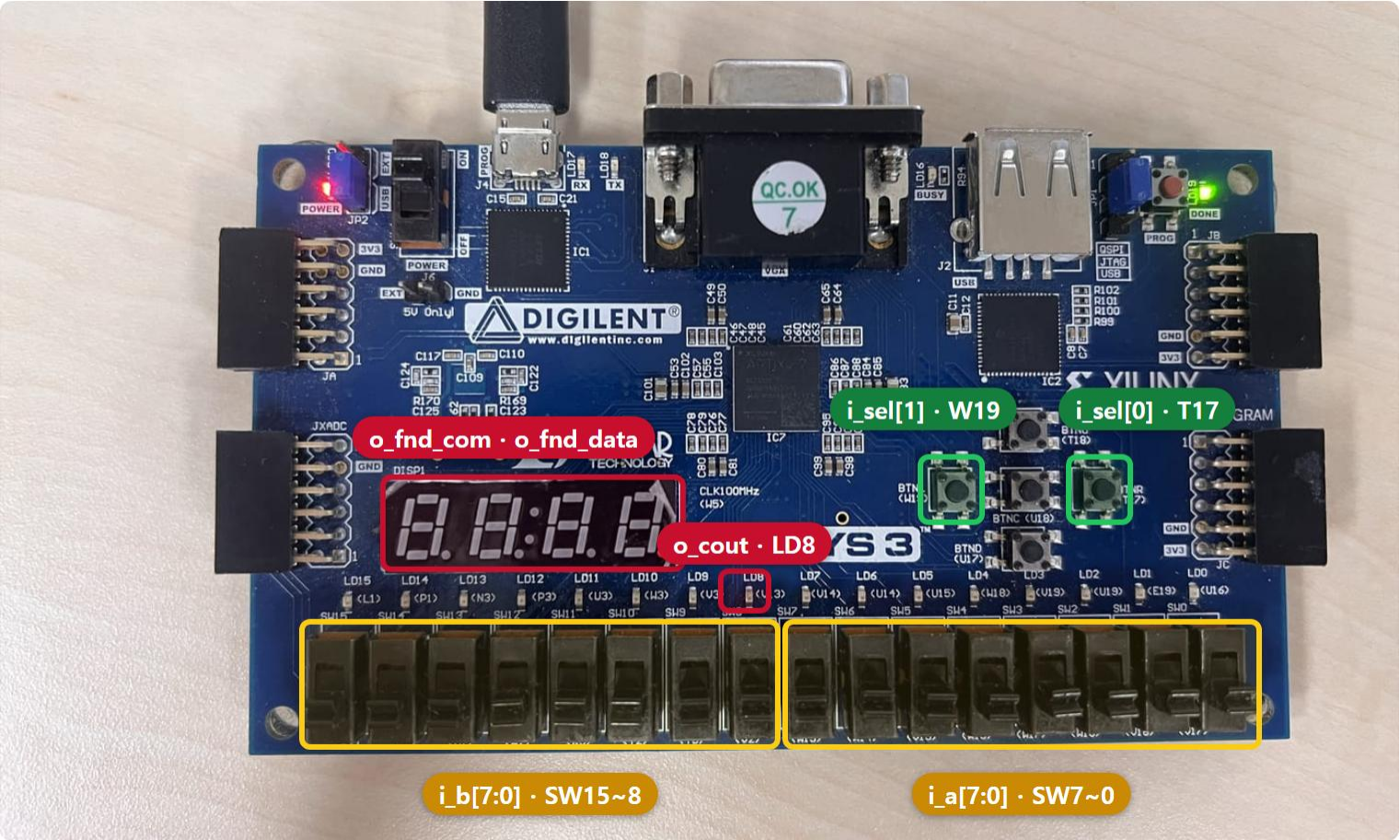
풀 때는 모든 FF가 같은 엣지에서 출발 (동기 장점)

이 설계에 적용 시

버튼 → 디바운스 → 동기화기 → 분주기·FSM·카운터

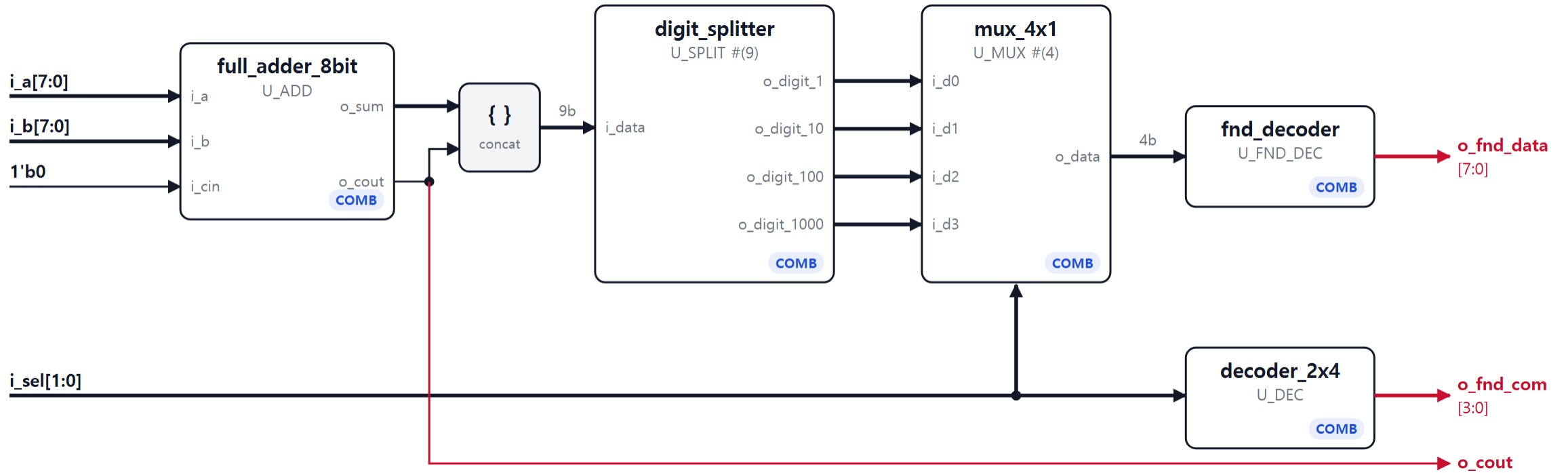
스위치 입력의 값 흔들림(p.31 개선 제안)도 같은 FF 2개 방식으로 해결

V1 I/O Information — top_adder_fnd_8bit



포트	보드	핀
i_a[7:0]	SW7~SW0	W13...V17
i_b[7:0]	SW15~SW8	R2...V2
i_sel[1]	BTNL	W19
i_sel[0]	BTNR	T17
o_fnd_com[3:0]	AN3~AN0	W4 V4 U4 U2
o_fnd_data[7:0]	DP, CG~CA	V7 U7...W7
o_cout	LD8	V13

V1 Block Diagram — Button-Selected Digit Display



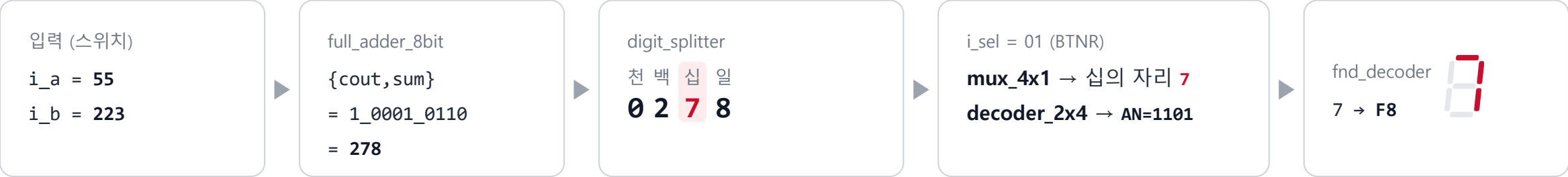
COMB 조합 회로

굵은 선 = 버스

빨간 선 = 최상위 출력

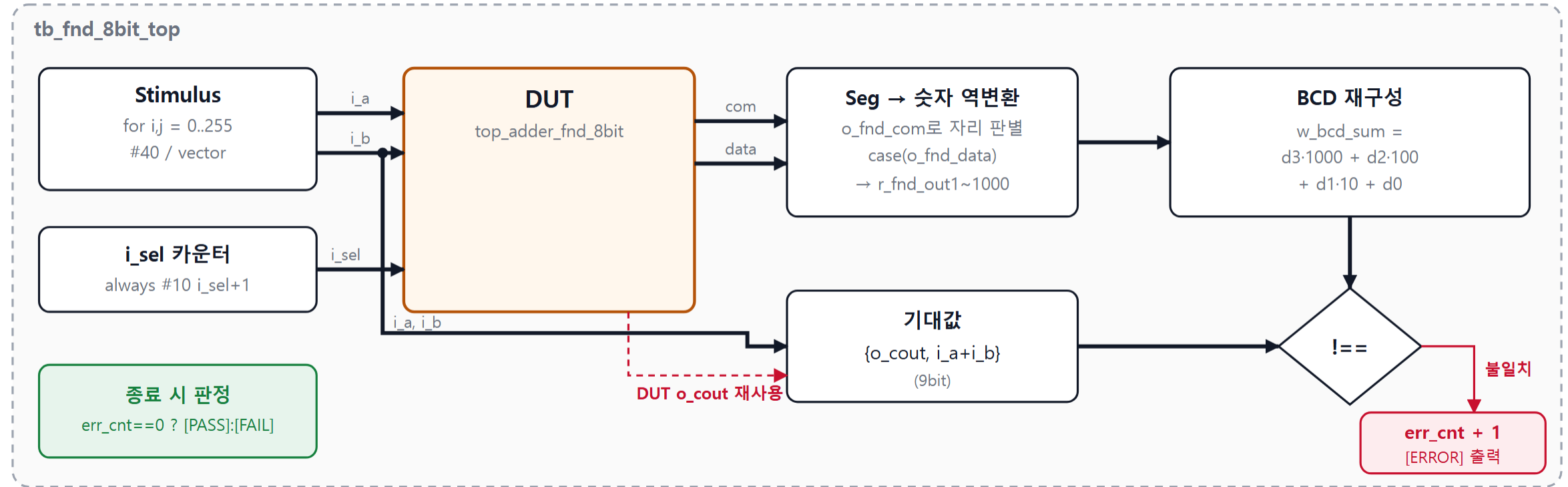
i_sel은 MUX와 Decoder에 동시에 들어가 "어느 자리를, 어떤 숫자로"를 일치시킴

Module Roles — Data Flow of 55 + 223 = 278



모듈	역할	구현 핵심
full_adder_8bit	8bit 덧셈, 최대 255+255 = 510 (9bit 필요)	4bit RCA × 2, {o_cout, o_sum}
digit_splitter	9bit 이진수 → 천·백·십·일 자리 BCD 4개로 분리	/10, %10 연산 (WIDTH=9)
mux_4x1	4개 자리 중 지금 표시할 숫자 하나 를 선택	case(i_sel) → i_d0~i_d3
decoder_2x4	MUX와 같은 i_sel로 그 숫자가 켜질 자리(Anode) 선택	00→1110 ... 11→0111 (one-cold)
fnd_decoder	BCD 숫자(0~9) → 7-segment 폰트(Active-Low)	0→C0, 1→F9 ... 9→90

65,536-Case Exhaustive Check — Self-Checking TB Decodes the Output Pins

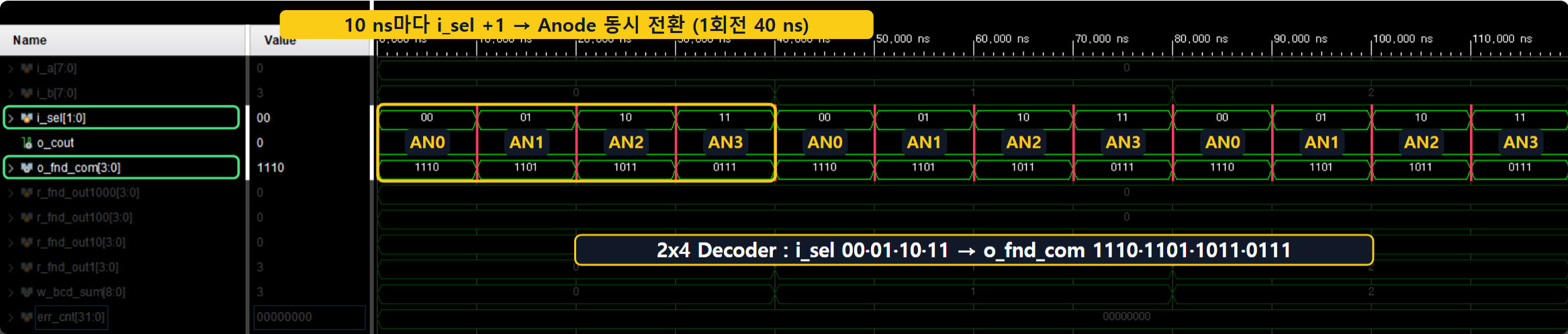


① 전수 자극 — i, j = 0~255 이중 루프, 벡터당 40 ns (= i_sel 4회전)

② 블랙박스 관측 — 내부 신호가 아닌 **FND** 핀 출력을 숫자로 역변환

③ 자동 판정 — 불일치 시 err_cnt + 1, 종료 시 **[PASS]/[FAIL]**

Exactly One Anode Turns On per i_sel Value — 2x4 Decoder Works

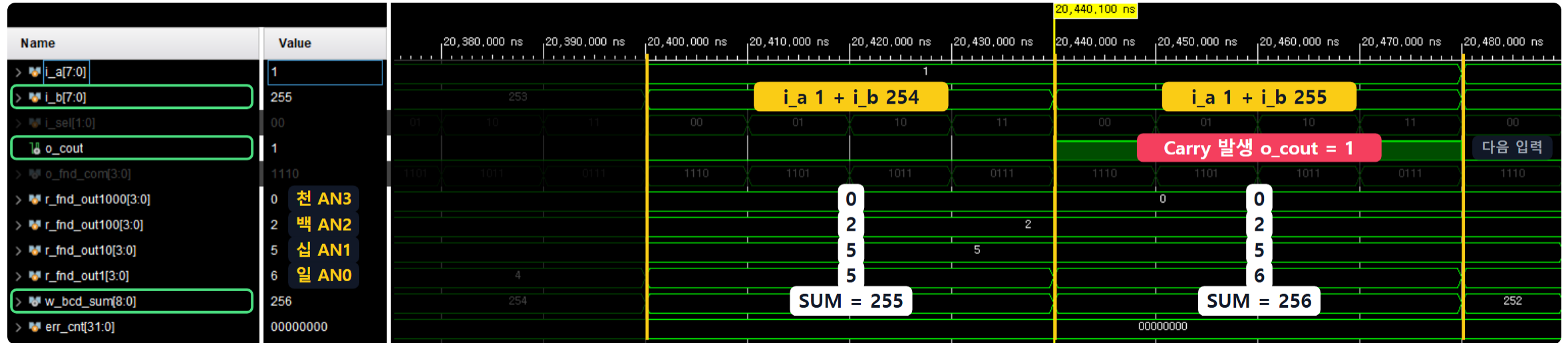


i_sel	o_fnd_com	켜지는 자리	표시 숫자 (mux)
00	1110	AN0 · 일	o_digit_1
01	1101	AN1 · 십	o_digit_10
10	1011	AN2 · 백	o_digit_100
11	0111	AN3 · 천	o_digit_1000

확인 — 10 ns ~ 110 ns 전 구간에서 i_sel 변화와 o_fnd_com 전환이 동시에 일어남 (조합 회로)

항상 한 자리만 0 (one-cold) → 두 자리가 동시에 켜지는 구간 없음

a + b = BCD Output — Every Digit Matches the Sum, Carry Range Included



```

2620800 ns : i_a = 255 / i_b = 240 o_cout = 1 BCD_NUM = 495
2620840 ns : i_a = 255 / i_b = 241 o_cout = 1 BCD_NUM = 496
2620880 ns : i_a = 255 / i_b = 242 o_cout = 1 BCD_NUM = 497
2620920 ns : i_a = 255 / i_b = 243 o_cout = 1 BCD_NUM = 498
2620960 ns : i_a = 255 / i_b = 244 o_cout = 1 BCD_NUM = 499
2621000 ns : i_a = 255 / i_b = 245 o_cout = 1 BCD_NUM = 490
2621010 ns : i_a = 255 / i_b = 245 o_cout = 1 BCD_NUM = 400
2621020 ns : i_a = 255 / i_b = 245 o_cout = 1 BCD_NUM = 500
2621040 ns : i_a = 255 / i_b = 246 o_cout = 1 BCD_NUM = 501
2621080 ns : i_a = 255 / i_b = 247 o_cout = 1 BCD_NUM = 502
2621120 ns : i_a = 255 / i_b = 248 o_cout = 1 BCD_NUM = 503
2621160 ns : i_a = 255 / i_b = 249 o_cout = 1 BCD_NUM = 504
2621200 ns : i_a = 255 / i_b = 250 o_cout = 1 BCD_NUM = 505
2621240 ns : i_a = 255 / i_b = 251 o_cout = 1 BCD_NUM = 506
2621280 ns : i_a = 255 / i_b = 252 o_cout = 1 BCD_NUM = 507
2621320 ns : i_a = 255 / i_b = 253 o_cout = 1 BCD_NUM = 508
2621360 ns : i_a = 255 / i_b = 254 o_cout = 1 BCD_NUM = 509
2621400 ns : i_a = 255 / i_b = 255 o_cout = 1 BCD_NUM = 500
2621410 ns : i_a = 255 / i_b = 255 o_cout = 1 BCD_NUM = 510
[PASS] : ALL BCD CORRECT!

```

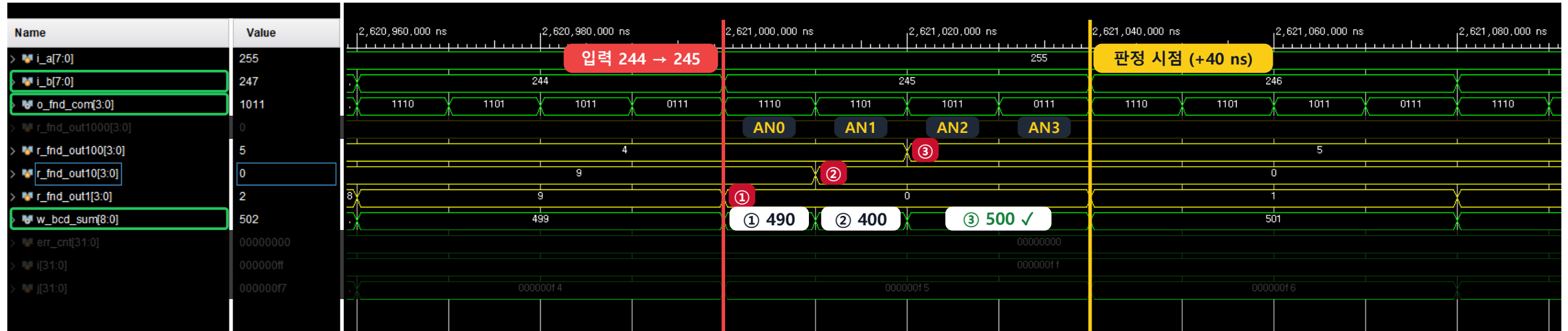
판독 — 1 + 254 → 천·백·십·일 = 0 2 5 5 = 255

1 + 255 → 천·백·십·일 0 2 5 6 = 256 → o_cout = 1

[PASS] ALL BCD CORRECT — 255 + 255 = 510까지 65,536 벡터 일치 (Tcl Console)

로그의 490 → 400 → 500은 입력이 바뀐 직후 자리가 하나씩 갱신되는 과도 구간 값. 판정은 40 ns 뒤에 하므로 오류 아님 → 해석은 다음 장

Log 490 → 400 → 500 Is a Transient — 500 at the Check Point



① 2,621,000 ns · AN0

일의 자리만 새 값(9→0)
백·십은 이전 입력값 유지

490 4 · 9 · 0

② 2,621,010 ns · AN1

십의 자리 갱신(9→0)
백은 아직 이전값 4

400 4 · 0 · 0

③ 2,621,020 ns · AN2

백의 자리 갱신(4→5)
이제 4자리 모두 새 값

500 5 · 0 · 0

④ 2,621,040 ns · 판정

입력 후 #40 = Anode 1바퀴
{o_cout, i_a+i_b} 와 비교

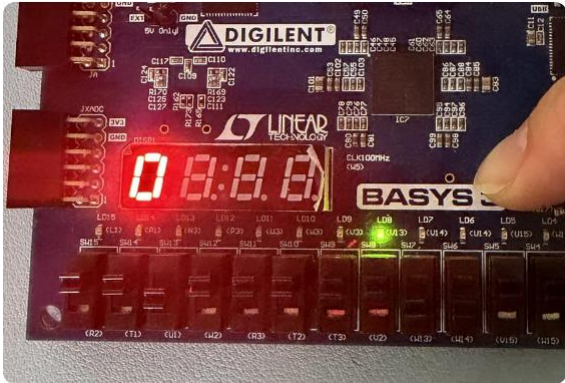
500 = 255 + 245 ✓ PASS

이렇게 찍히는 이유

— TB는 o_fnd_com이 가리키는 자리 하나만 역변환해 r_fnd_out*에 저장. 입력이 바뀌면 AN0 → AN3 순서로 10 ns마다 한 자리씩 새 값이 되고, \$monitor는 w_bcd_sum이 바뀔 때마다 출력하므로 중간값 490 · 400이 로그에 남음.

판정 — 4자리가 모두 갱신된 #40 시점에만 비교 → 과도값은 판정 대상이 아님 (65,536 벡터, err_cnt = 0)

Board Test — $55 + 223 = 278$, One Digit per Button Combination



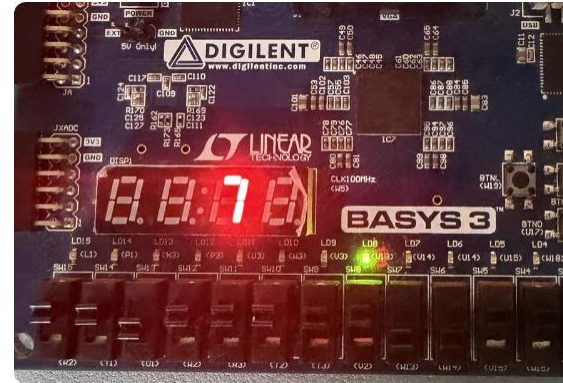
$i_sel = 11$
BTNL + BTNR → AN3 · 천



$i_sel = 10$
BTNL → AN2 · 백



$i_sel = 01$
BTNR → AN1 · 십



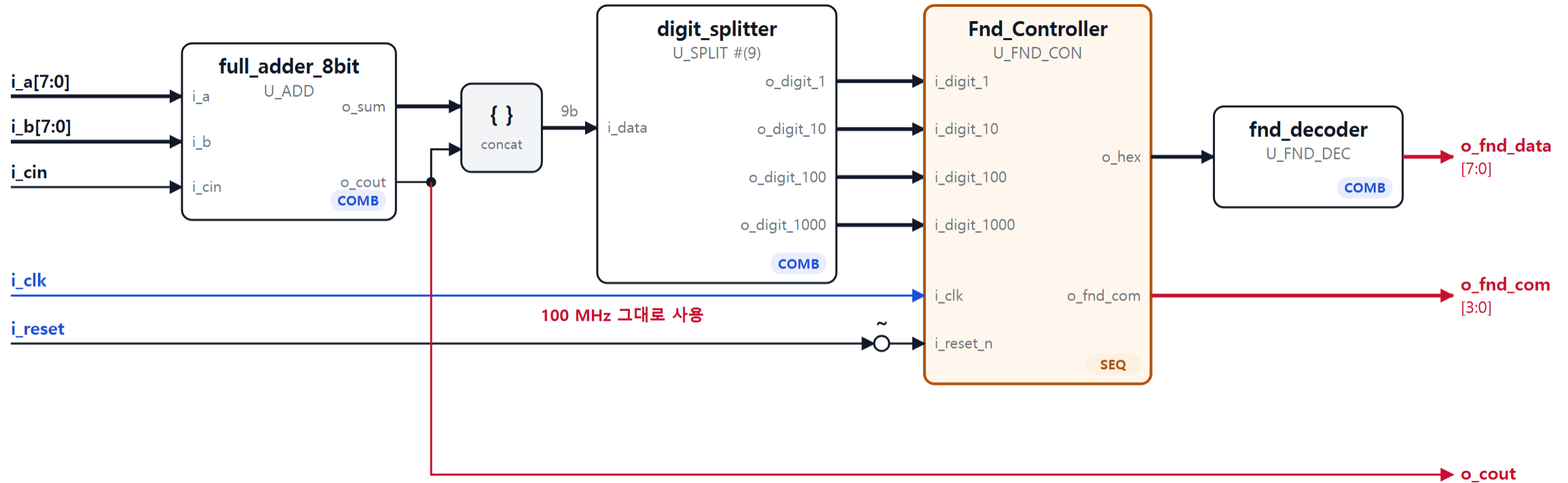
$i_sel = 00$
누르지 않음 → AN0 · 일



판독 — 4장을 합치면 278 ✓
 $278 \geq 256$ 이므로 LD8(o_cout) 점등 ✓

트러블슈팅 #1 — W19·T17 버튼을 매번 눌러 Anode를 바꿔야 하므로 합을 한눈에 읽을 수 없음

Fix #2 — FSM Replaces MUX and Decoder, Anodes Cycle Automatically

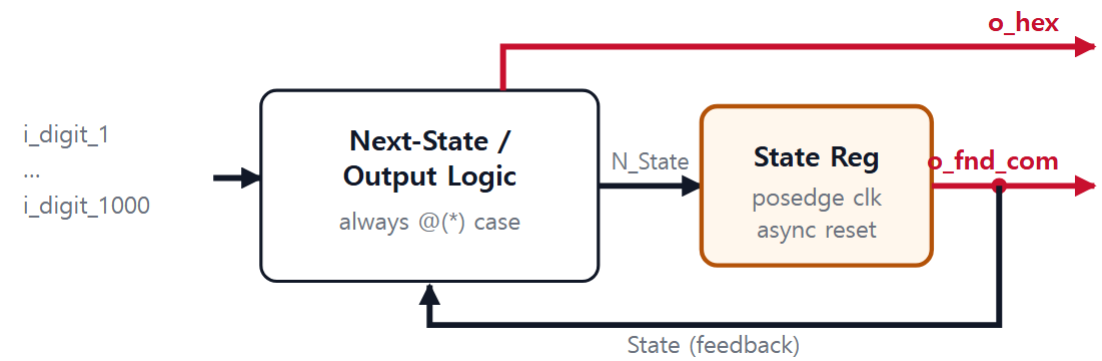
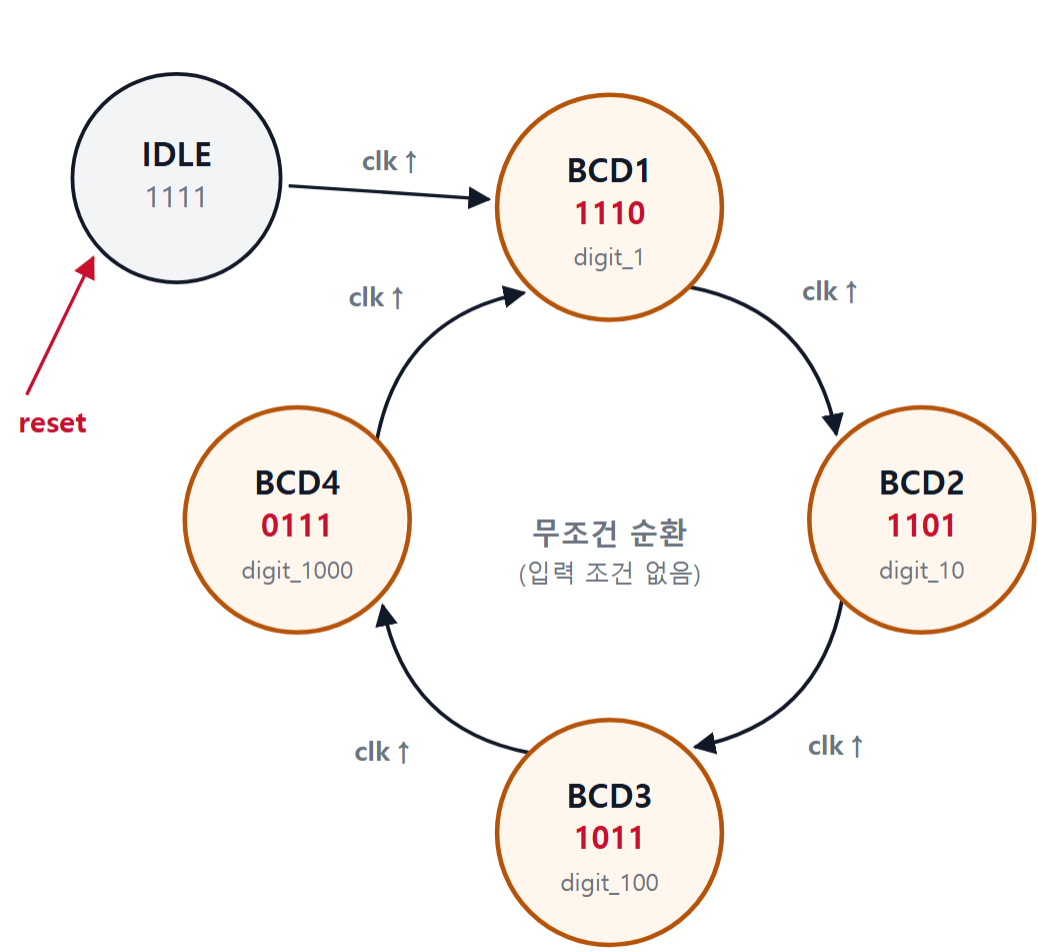


COMB 조합 회로

SEQ 순차 회로 (신규)

버튼 입력 i_sel 제거 → 사람 대신 FSM이 자리 선택

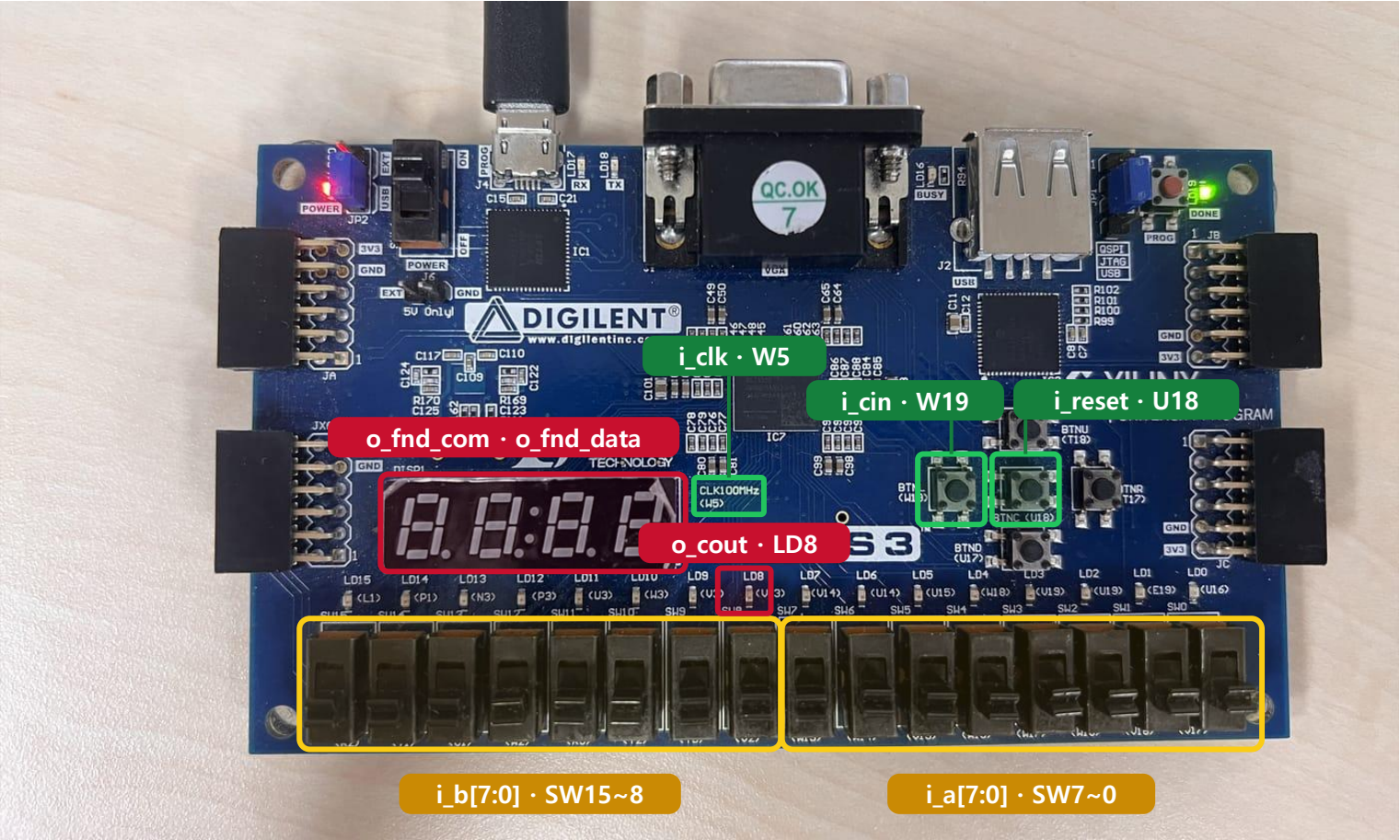
Inside Fnd_Controller — 4-State FSM Whose State Value Is the Anode Output



State	값 = o_fnd_com	o_hex	다음
IDLE	1111 (모두 OFF)	4'hF	BCD1
BCD1	1110 (AN0)	i_digit_1	BCD2
BCD2	1101 (AN1)	i_digit_10	BCD3
BCD3	1011 (AN2)	i_digit_100	BCD4
BCD4	0111 (AN3)	i_digit_1000	BCD1

설계 포인트 — State 인코딩 = Anode 패턴(one-cold)이라 출력 디코딩 로직이 없음 → Anode에 조합 글리치 없음. 출력이 State에만 의존하므로 **Moore형** 구조

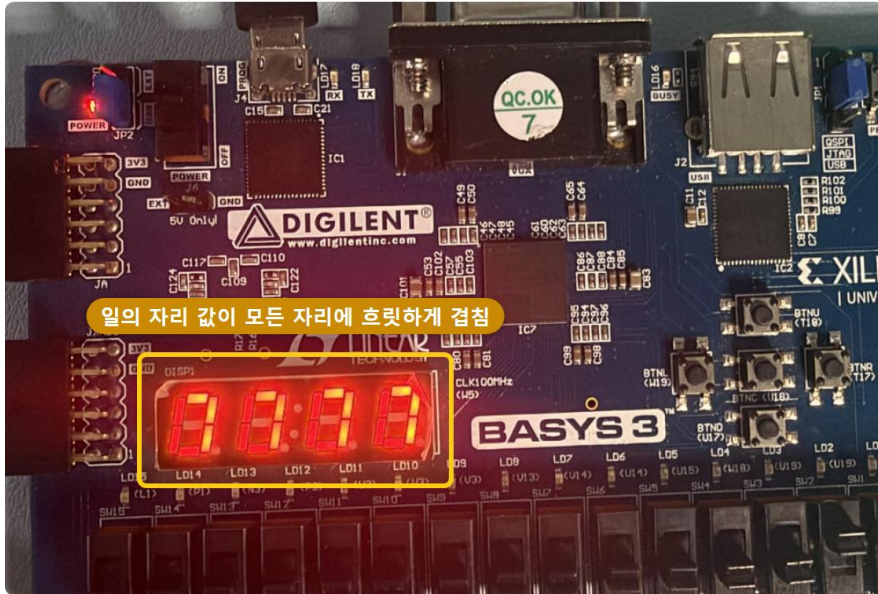
v2 I/O Ports — Clock, Reset, Carry-in Added; Digit-Select Buttons Removed



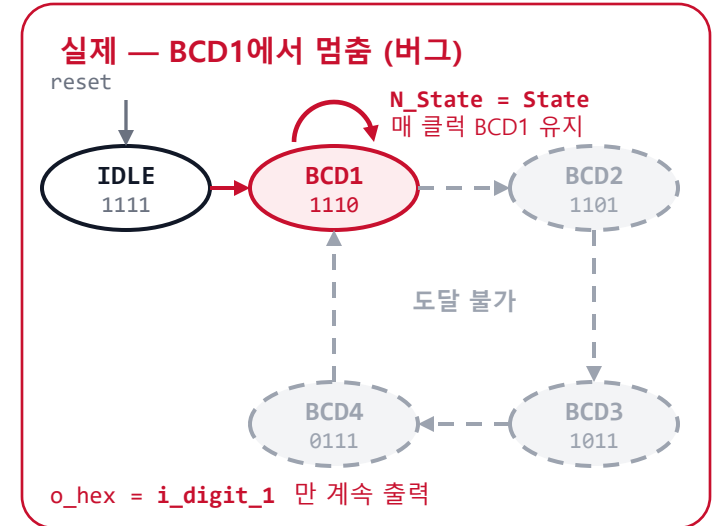
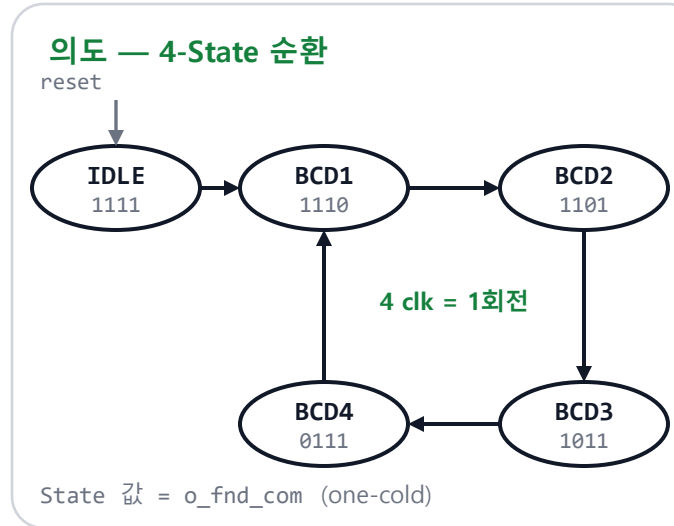
포트	보드	핀
i_clk	100 MHz OSC	W5
i_reset	BTNC	U18
i_cin	BTNL	W19
i_a / i_b	SW7~0 / SW15~8	v1과 동일
o_fnd_com / data	AN / Cathode	v1과 동일
o_cout	LD8	V13

Reset을 Active-High로 사용한 이유
BTNC(U18)는 누를 때 1 → Top에서 반전 (~i_reset_n)하여 하위 모듈의 Active-Low 리셋에 연결

Board Shows Only the Ones Digit — FSM Next-State Bug Locks State at BCD1



V2 첫 보드 출력 — TB 확인 없이 합성·다운로드한 결과

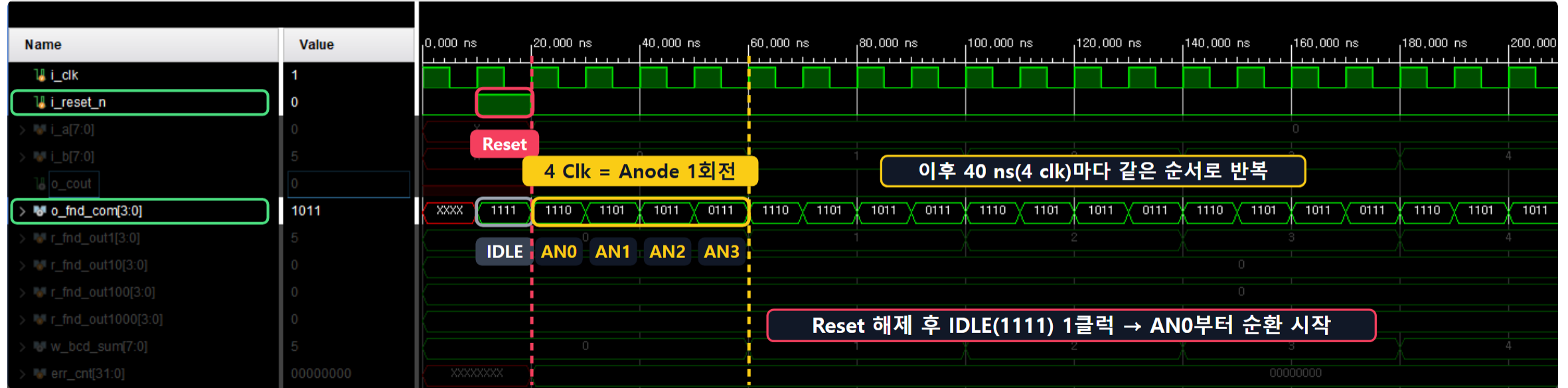


원인 — Next-State 로직에서 항상 **N_State = State**가 적용되어 case 문의 천이가 반영되지 않음 → IDLE 다음 BCD1에서 벗어나지 못함 (단순 코딩 실수)

교훈 — TB를 먼저 돌리지 않고 보드부터 출력함. 파형에서 o_fnd_com이 1110에 멈춘 것이 바로 보이므로, TB를 선행했다면 보드 출력 전에 걸러낼 수 있었던 오류

수정 후 재검증 — V1과 동일한 Self-checking TB: o_fnd_com으로 자리 판별 → o_fnd_data 역변환 → BCD_NUM 복원 → i_a+i_b+i_cin과 비교 (i_cin 0/1) → 결과 p.22 ~ 24

After Reset Release, IDLE → AN0~AN3 Cycle Once Every 4 Clocks

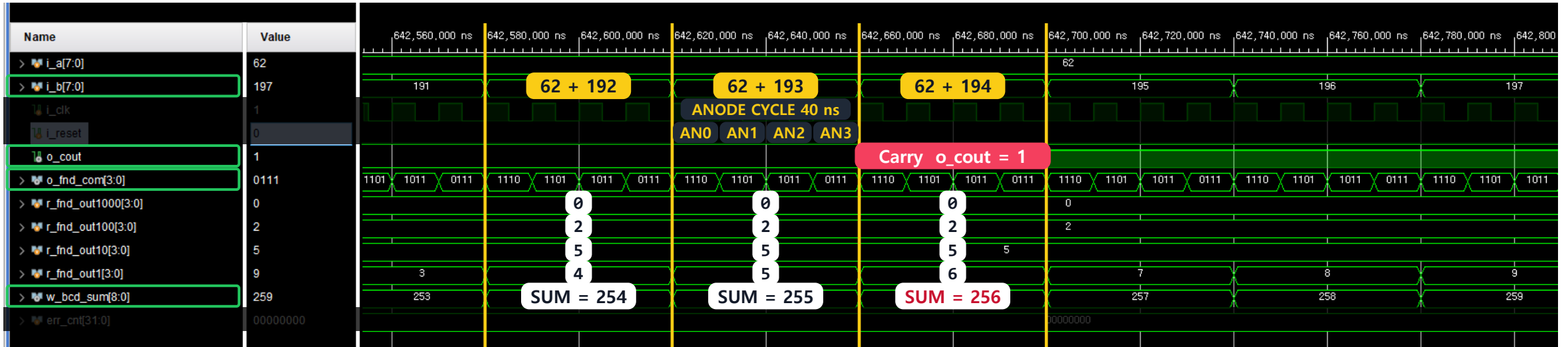


Reset — 버튼(BTNC) High 동안 State = IDLE(1111), 모든 Anode OFF

순환 — 1110 → 1101 → 1011 → 0111, 클럭마다 한 자리만 ON

Active-High Reset — BTNC(U18)는 누를 때 1 → Top에서 반전해 하위 모듈에 연결

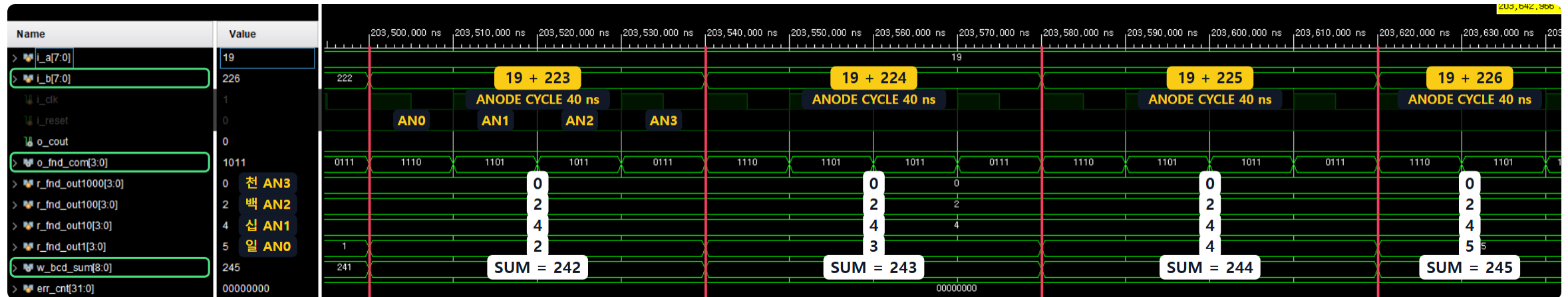
Each Anode Step Updates Its Digit — Correct Through the 255 → 256 Carry



판독 $62 + 192 \rightarrow 0\ 2\ 5\ 4 = 254$ $62 + 193 \rightarrow 0\ 2\ 5\ 5 = 255$
 $62 + 194 \rightarrow 0\ 2\ 5\ 6 = 256 \rightarrow \text{o_cout} = 1$ (256부터 Carry)

TB 수정 — w_bcd_sum이 입력 구간과 같은 시점에 갱신되도록 정렬 → 이전 캡처의 "한 구간 늦게 표시" 해소, 파형의 SUM이 곧 그 입력의 결과

One Anode Cycle per Input — All 131,072 Outputs Checked



```

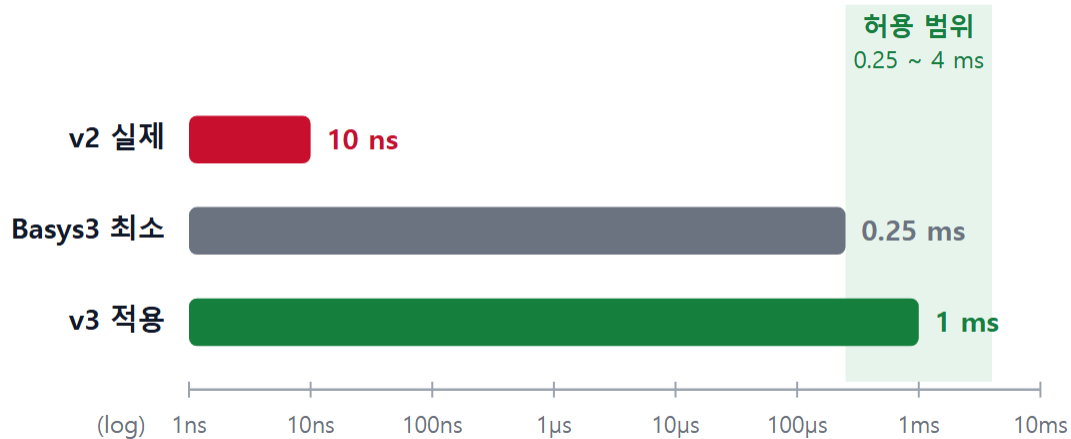
5242440 ns : i_a = 255 / i_b = 244 o_cout = 1 BCD_NUM = 500
5242460 ns : i_a = 255 / i_b = 245 o_cout = 1 BCD_NUM = 501
5242500 ns : i_a = 255 / i_b = 246 o_cout = 1 BCD_NUM = 502
5242540 ns : i_a = 255 / i_b = 247 o_cout = 1 BCD_NUM = 503
5242580 ns : i_a = 255 / i_b = 248 o_cout = 1 BCD_NUM = 504
5242620 ns : i_a = 255 / i_b = 249 o_cout = 1 BCD_NUM = 505
5242660 ns : i_a = 255 / i_b = 250 o_cout = 1 BCD_NUM = 506
5242700 ns : i_a = 255 / i_b = 251 o_cout = 1 BCD_NUM = 507
5242740 ns : i_a = 255 / i_b = 252 o_cout = 1 BCD_NUM = 508
5242780 ns : i_a = 255 / i_b = 253 o_cout = 1 BCD_NUM = 509
5242820 ns : i_a = 255 / i_b = 254 o_cout = 1 BCD_NUM = 500
5242830 ns : i_a = 255 / i_b = 254 o_cout = 1 BCD_NUM = 510
5242860 ns : i_a = 255 / i_b = 255 o_cout = 1 BCD_NUM = 511
[PASS] : i_cin = 1 -> ALL BCD CORRECT!
[PASS] : ALL BCD CORRECT! (i_cin = 0, 1)

```

4자리 값을 숫자로 환산해 비교, 틀리면 **err_cnt + 1** → 끝까지
err_cnt = 0, 131,072 케이스 (i_{cin} 0/1 각 65,536) **[PASS]**

결론 — FSM 수정(BCD1 고정 해소) 후 BCD 값 전수 일치 → 남은
 고스팅의 원인은 **보드 구동 타이밍**

10 ns per Digit — 25,000× Shorter Than the Hardware Needs



100 MHz로 Anode를 바꾸면 PNP 트랜지스터·LED가 꺼지기 전에 다음 자리가 켜짐
→ 특정 자리의 숫자 잔상이 옆자리에 흐릿하게 겹침 (고스팅)

해결 방안 — 100 MHz 클럭 분주 → 1 kHz (주기 1 ms)

카운터 값	출력
0 ~ 49,999	1
50,000 ~ 99,999	0
100,000 클럭 = 1 ms	→ 반복

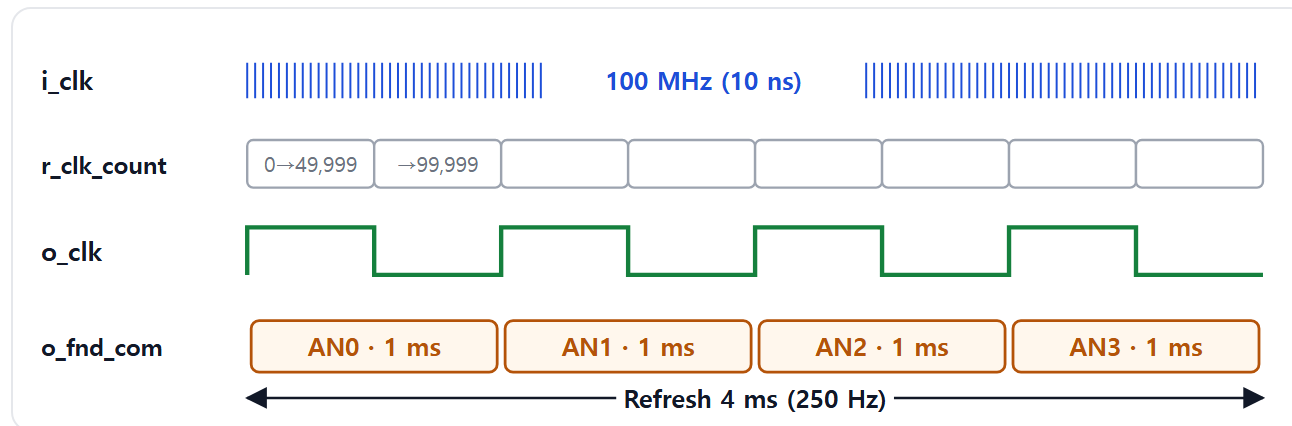
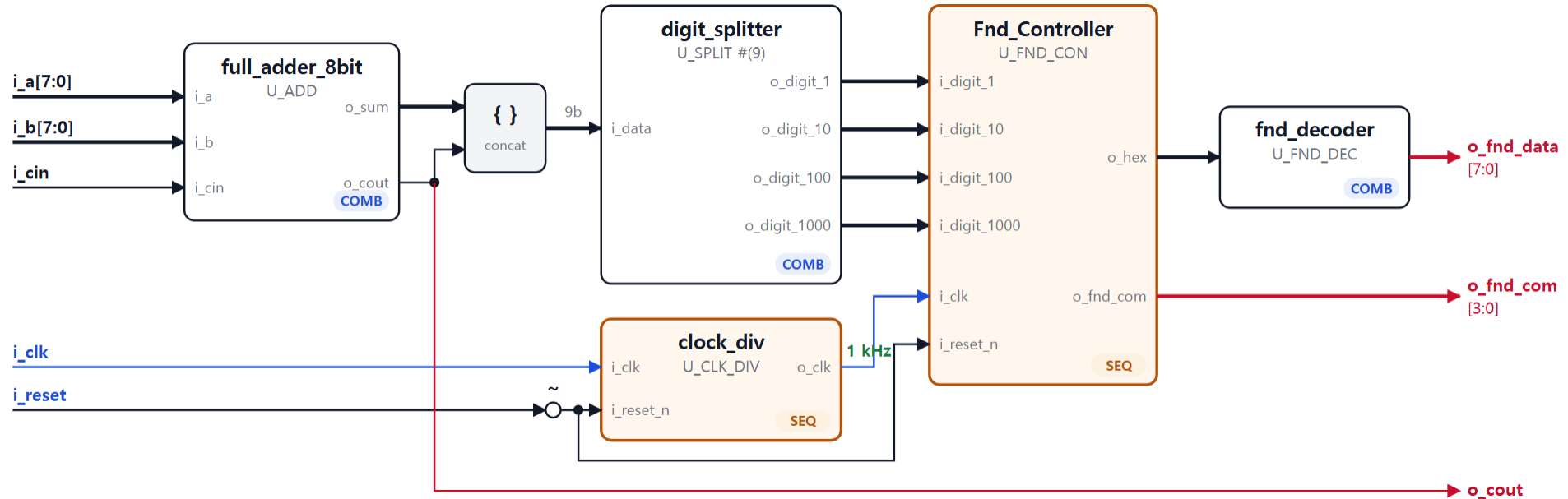
1 ms

자리당 점등 (≥ 0.25 ms ✓)

4 ms

화면 갱신 (1~16 ms ✓)

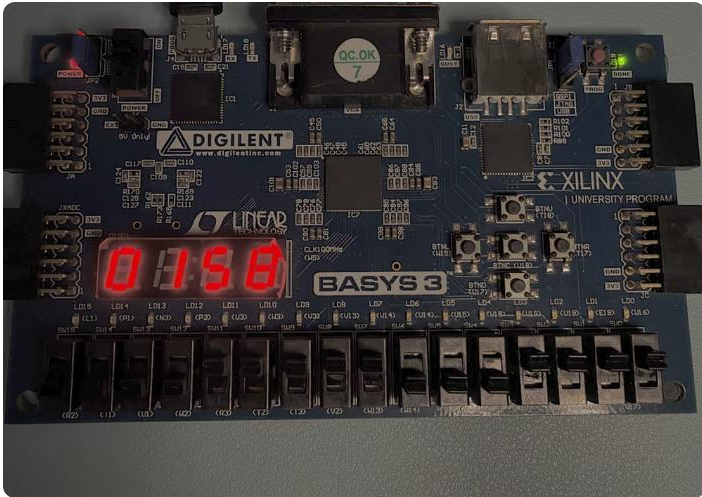
clock_div Added — FSM Runs at 1 kHz, Each Digit Lit for 1 ms



clock_div

- 20bit 카운터 r_clk_count로 100 MHz 클럭 계수
- 50,000 미만 → 1, 이상 → 0 : **Duty 50 %, 1 kHz**
- 출력 o_clk_100Hz가 Fnd_Controller의 클럭으로 입력

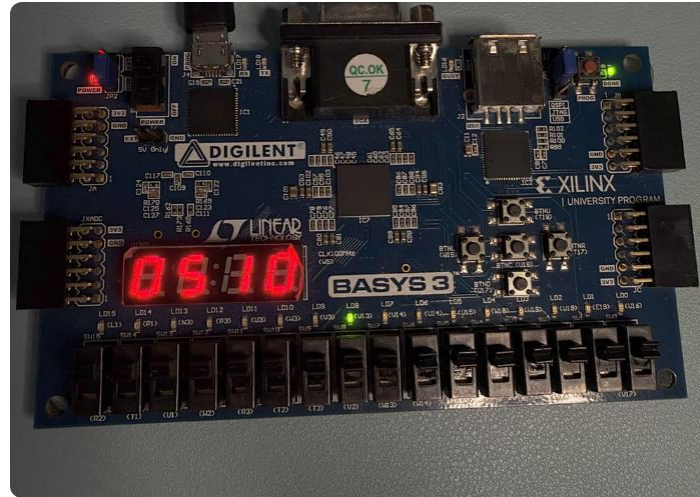
v3 Board Test — All 4 Digits Shown at Once, No Ghosting



FND 표시

0158

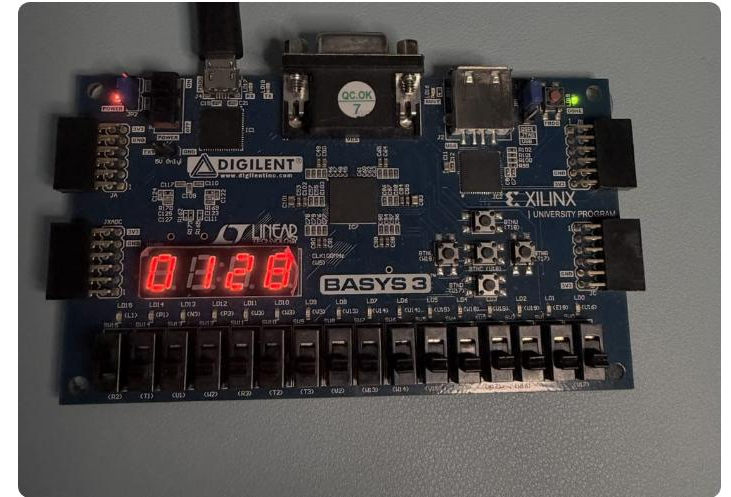
$i_a = 143$ $i_b = 15$
 $o_cout = 0$ · 합 < 256



FND 표시

0510

$i_a = 255$ $i_b = 255$
 $o_cout = 1$ (LD8 ON) · 최대값 510



FND 표시

0128

$i_a = 0$ $i_b = 128$
 $o_cout = 0$ · 합 < 256

확인 — 3개 입력 모두 $i_a + i_b$ 와 일치 · 버튼 조작 없이 합이 바로 읽힘 · 최대값 510에서만 Carry LED(LD8) 점등 · 1 kHz 스캔으로 깜빡임·잔상 없음

Assessment — Strong Exhaustive Self-Check, Gap in Output-Protocol Checks

평가 항목	판정	근거
입력 공간 커버리지	우수	clk TB: i_cin 0/1 × 256 × 256 = 131,072 전수 자극 (v1 TB 65,536, i_cin 고정)
자동 판정 (Self-checking)	우수	err_cnt 누적 + 종료 시 [PASS]/[FAIL], 오류 시 입력값과 표시값을 함께 출력
관측 지점	우수	내부 신호가 아닌 FND 핀 출력 을 역변환 → 실제 보드가 보여줄 값을 검증
기대값 독립성	양호	clk TB는 i_a+i_b+i_cin으로 독립 계산 → 덧셈 올림 고장(버그 ①) 검출. 단 LED 출력은 미비교
출력 동작 확인 (Anode)	미흡	한 자리씩·순서대로·4자리 모두 켜지는지 확인 없음 → 버그 ②④ 미검출 (p.29)
검증 대상 일치	보완 필요	보드에 올린 v3(clock_div 포함)가 아닌 _test Top만 시뮬레이션
판정 신뢰성	보완 필요	고정 #40 판정, 입력 변경·리셋 해제·판정이 clk edge와 같은 시각(실행 순서 의존), 복원 9bit
문제 해결 절차	양호	보드 이상 → TB로 RTL 확인 → 물리 타이밍으로 원인 축소. 단 TB 없이 보드 출력이 먼저여서 FSM 버그(BCD1 고정)를 사전에 못 거름

종합 (우수 5 · 양호 4 · 보완 3 · 미흡 2)

31 / 40

B — Anode 확인 보강 필요

강점 — 전수 입력 + 핀 레벨 역변환 + 자동 판정. 보드 이상을 TB로 RTL과 분리한 문제 해결 절차

핵심 약점 — 화면이 한 자리씩, 순서대로, 4자리 모두 켜지는지 확인하지 않음

Gaps in the Original TB — Misses 2 of 5 Injected Bugs, 1 Partly

방법 회로에 버그를 하나씩 넣고 같은 입력 131,072개를 돌려, TB가 FAIL을 내는지 확인 · 보드 화면 예시는 255 + 255 = 510

심은 버그	보드 화면 (정상이면 0510)	기존 TB	새 TB
정상 회로	0510	PASS	PASS
① 덧셈 올림(Carry) 고장	0254 510 → 254, LED 꺼짐	✓ 잡음	✓ 잡음
② 천의 자리가 안 켜짐	8510 맨 앞 0이 안 보임	✗ 놓침	✓ 잡음
③ 첫 자리에서 멈춤	8880 일의 자리만 보임	✓ 잡음	✓ 잡음
④ 천의 자리 순서 건너뛰	8510 3자리만 돌아감	✗ 놓침	✓ 잡음
⑤ 두 자리가 동시에 켜짐	0510 일의 자리에 십의 자리가 겹침	△ 일부 놓침	✓ 잡음

놓침 이유 기존 TB는 '최종 숫자 값'만 비교. 천의 자리는 원래 0이라 꺼져서 안 보여도 값(510)은 동일 → PASS

6 Gaps in the Original TB → How the New TB Covers Each

기존 TB가 보지 않던 것		새 TB가 확인하는 것	잡는 버그
1	4자리가 다 켜졌는지 안 봄	입력마다 4자리를 전부 다시 읽은 뒤에만 채점	② ③ ④ ⑤
2	켜지는 순서를 안 봄	AN0 → AN1 → AN2 → AN3 순서를 매번 확인	② ④ ⑤
3	두 자리가 동시에 켜져도 모름	한 번에 한 자리만 켜지는지 확인	② ⑤
4	LED(Carry) 출력은 안 봄	LED 값도 정답과 따로 비교	①
5	이전 입력의 값이 남아 있어도 통과	입력이 바뀌면 기록을 지우고 처음부터 다시 읽음	② ④
6	40 ns마다 무조건 채점	4자리를 다 읽은 순간 채점 → 스캔 속도가 달라도 그대로 사용	보드 설정

그대로 둔 장점 i_cin 0/1 포함 131,072개 전부 입력 · 정답은 입력만으로 계산 (회로 출력을 정답에 쓰지 않음)

Improvements

검증 (TB) — 현재 clk TB 상태 → 새 TB

반영됨

독립 기대값 + i_cin 전수

i_a+i_b+i_cin, 131,072 전수 (사용자 수정) → 버그 ① 검출. LED는 새 TB에서 따로 비교

미흡

Anode 프로토콜 확인

한 자리씩 · 순서대로 · 4자리 모두 켜지는지 확인 → 새 TB에서 버그 ② ④ ⑤ 검출

보완 필요

복원 폭 · 무효 폰트

읽은 값을 넉넉한 폭(14bit)으로 복원, 0~9 이외 모양은 즉시 오류 → 새 TB 구현

RTL (설계 관점)

보완 필요

분주 클럭을 클럭으로 사용

로직으로 만든 클럭이 FSM을 구동 → 전용 클럭망(BUFG) 미사용, 클럭 도메인 증가. 100 MHz 단일 클럭 + 1 kHz Clock Enable 권장

양호

비동기 입력 처리

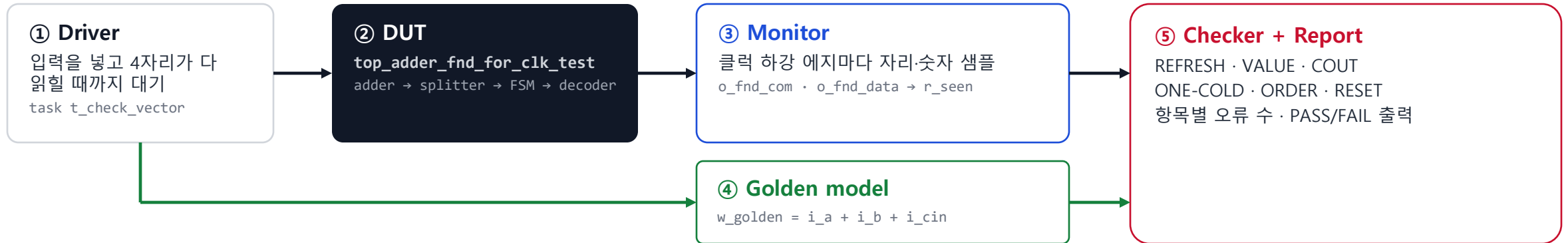
고찰의 “값 흔들림”은 2-FF 동기화 + 디바운스, 프레임 경계 래치로 해결 가능

적용 우선순위

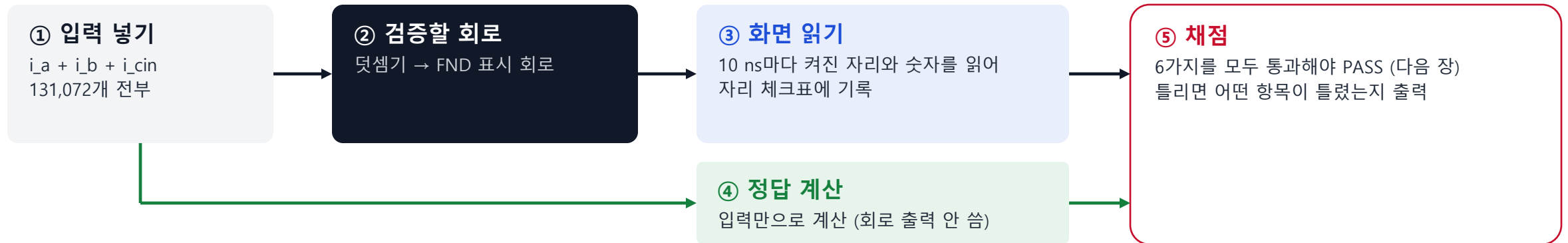
- 1 Anode 순서·갱신 확인 — 새 TB로 버그 ② ④ ⑤ 검출 (p.29~30)
- 2 V3 최종 Top 시뮬레이션 — 같은 TB의 V3 모드로 보드와 같은 구조 검증
- 3 Clock Enable 구조 전환 — 단일 클럭 도메인으로 타이밍 분석 단순화

New TB Structure — Module Layout and Plain Explanation

TB 모듈 구성 (코드 기준)



쉽게 말하면 — 같은 번호끼리 같은 역할



1 ms 대신 10 ns를 쓴 이유 보드는 자리당 1 ms(1 kHz)로 스캔하지만, 그 속도로 131,072개를 전부 돌리면 시뮬레이션에 오래 걸림(Vivado 시뮬레이터 속도로 환산 시 4.9시간). 그래서 분주기를 뺀 시뮬레이션용 Top(자리당 10 ns, 약 10초)으로 전수 검증.

6 Scoring Checks — Values Plus How the Display Lights Up

1 한 번에 한 자리만 ON

두 자리가 같이 켜지면 숫자가 겹쳐 보임

예) 1110 ✓

예) 1100 ✗

잡는 버그 ② ⑤

2 AN0 → AN1 → AN2 → AN3 순서 유지

자리를 건너뛰거나 거꾸로 돌면 오류

예) AN2 → AN3 ✓

예) AN2 → AN0 ✗

잡는 버그 ② ④ ⑤

3 제한 시간 안에 4자리 모두 ON

안 켜지는 자리, 멈춘 자리를 찾아냄

예) 40 ns 안에 4칸 ✓

예) 못 채움 ✗

잡는 버그 ② ③ ④ ⑤

4 화면 숫자 = 정답

4자리를 다 읽은 뒤 정답과 비교

예) 0510 = 255 + 255 ✓

잡는 버그 ①

5 LED(Carry) = 정답의 올림

합이 256 이상일 때만 켜져야 함

예) 510 → LED ON ✓

예) 254 → LED OFF ✓

잡는 버그 ①

6 리셋 중 4자리 모두 OFF

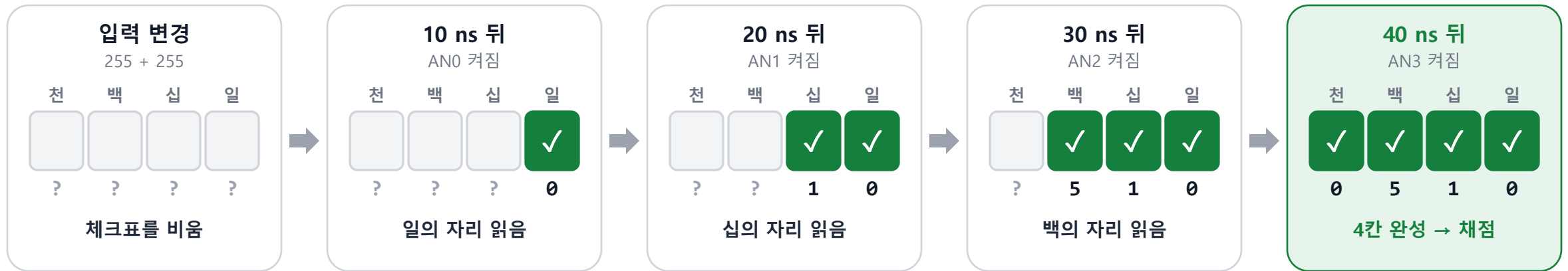
리셋 버튼을 누르는 동안 4자리 OFF

예) 1111 (모두 OFF) ✓

하나라도 틀리면 어떤 항목이, 어떤 입력에서 틀렸는지 출력 (항목별 최대 5줄) → 버그 위치를 바로 좁힐 수 있음

Scoring One Input — Score Once All 4 Checklist Cells Are Filled

자리 체크표 (코드 이름 r_seen) 입력이 바뀐 뒤 실제로 읽은 자리에 ✓를 표시하는 4칸짜리 표. 4칸이 다 차야 '화면 전체를 봤다'로 인정



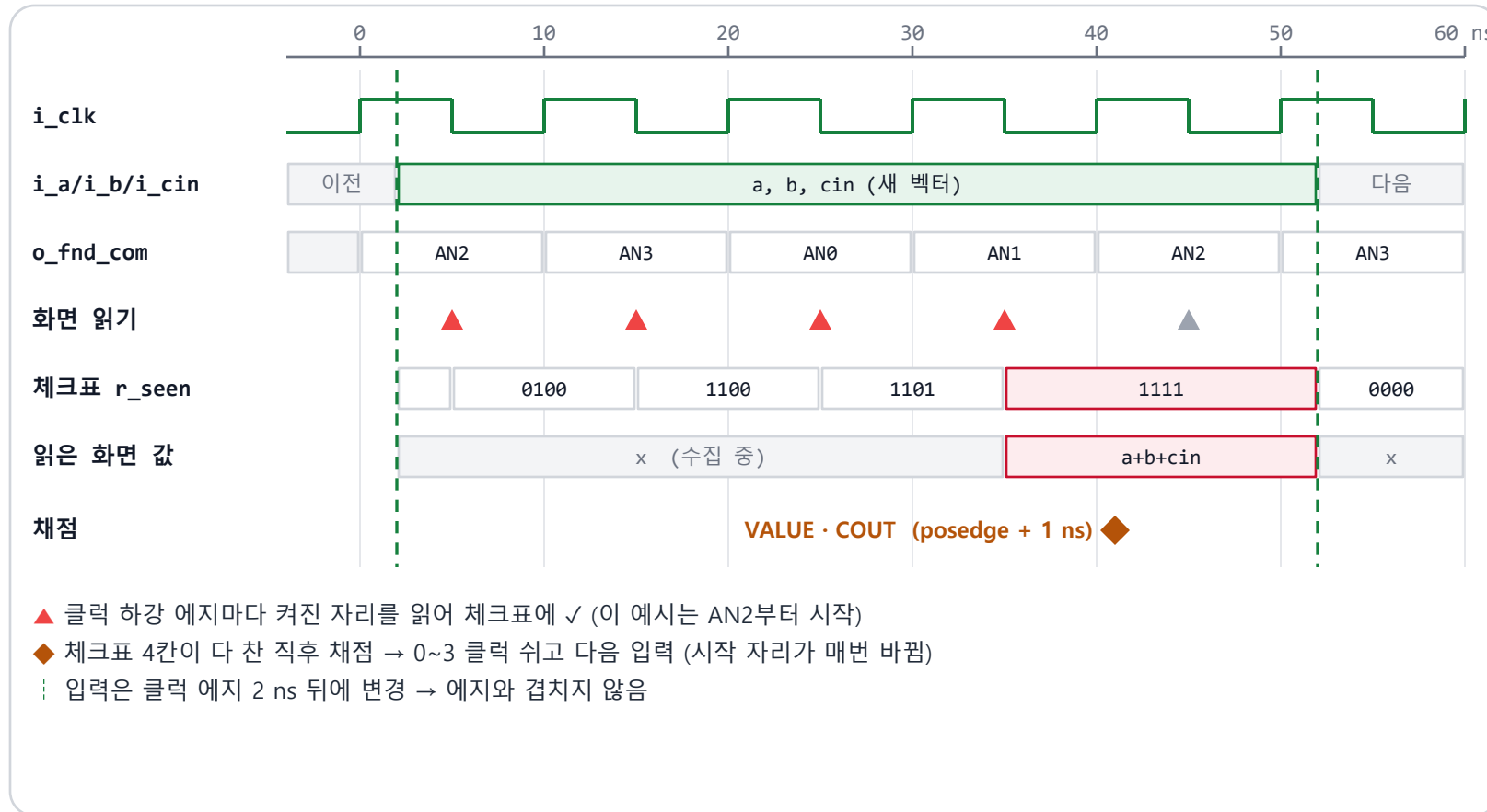
4칸이 다 차면 → 채점

화면 숫자 0510 = 정답 255 + 255 ✓ · LED = 올림 1 ✓
시작 자리와 무관하게 4칸 완성 시 채점 → 다음 입력으로

제한 시간 안에 안 차면 → '자리 누락' 오류

천의 자리가 안 켜지는 버그 ②, 첫 자리에서 멈추는 버그 ③을 이렇게 잡음
순서가 틀리거나 두 자리가 같이 켜지면 읽는 즉시 오류

Timing — Inputs Avoid Clock Edges, Read on Falling Edges, Score at 4 Cells



시작 자리가 달라도 되는 이유

판정은 “몇 ns 뒤”가 아니라 “4자리가 다 모였을 때”. 입력이 AN0가 아닌 자리에서 바뀌어도, 1 ms 스캔(V3)이어도 같은 코드로 판정.

그림 읽는 법

o_fnd_com 지금 켜진 자리

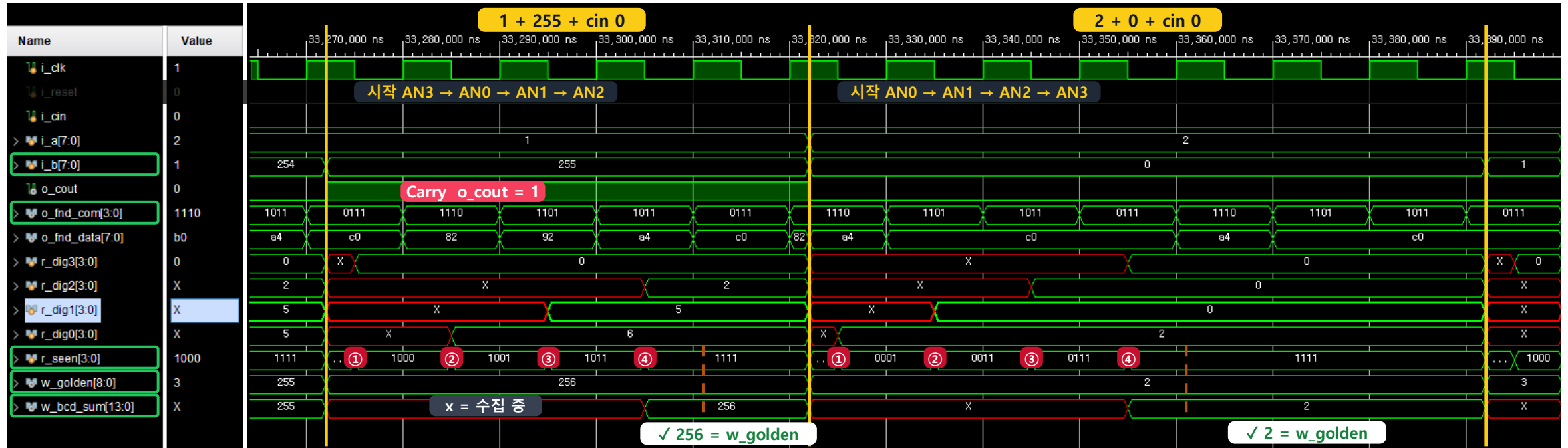
화면 읽기 ▲ 켜진 자리의 숫자를 읽는 순간

체크표 천·백·십·일 중 읽은 자리 = 1
 0100 → 1100 → 1101 → 1111

읽은 화면 값 4칸이 다 차기 전엔 x

채점 ◆ 화면 값 · LED를 정답과 비교

New TB Run — 131,072 Exhaustive PASS, Scoring After 4-Digit Capture



Tcl Console — Vivado 2020.2

```
vectors = 131072
[i_cin=0] PASS (0) [i_cin=1] PASS (0)
VALUE=0 COUT=0 REFRESH=0 ONE-COLD=0 ORDER=0 FONT=0 RESET=0
[PASS] : 131072 vectors, ALL BCD CORRECT! (i_cin = 0, 1)
```

V3 Top (define V3, clock_div 1 kHz 그대로): 대표 입력 7개 PASS — xsim 재현

파형 읽는 법

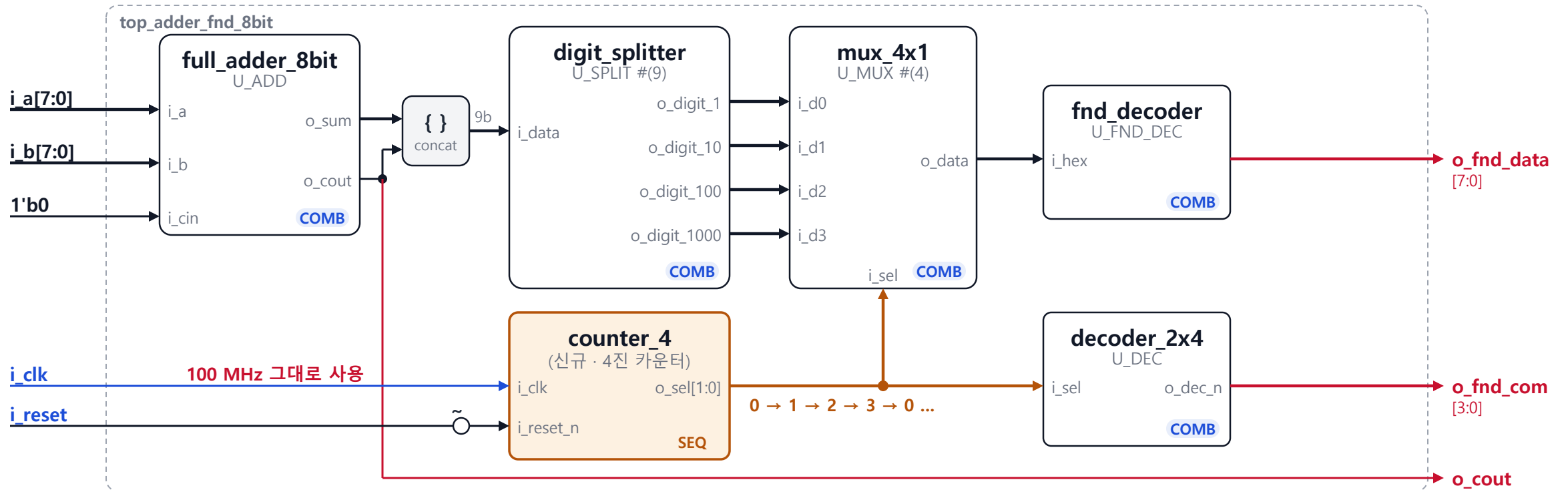
입력 변경(노랑) → 자리 체크표(r_seen)를 비우고 처음부터 다시 읽음

①~④ 화면 읽기 → 커진 자리를 읽어 체크표에 √. 1+255는 AN3부터, 2+0은 AN0부터 시작해도 똑같이 동작

체크표 4칸 완성 → 주황 점선에서 채점: 화면 값 = 정답 (256 = 256, 2 = 2)

0~3 클럭 쉬고 다음 입력 → 시작 자리가 매번 바뀜

Counter Scan — A Counter Replaces the Buttons, but 100 MHz Is Too Fast



COMB 조합 회로 SEQ 순차 회로(신규) 주황 선 = 버튼 대신 카운터가 만든 자리 번호

① 4진 카운터 = 자리 번호

클럭마다 $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$ 을 반복하는 2bit 카운터. 버튼 대신 '지금 켜 자리 번호'를 자동 생성

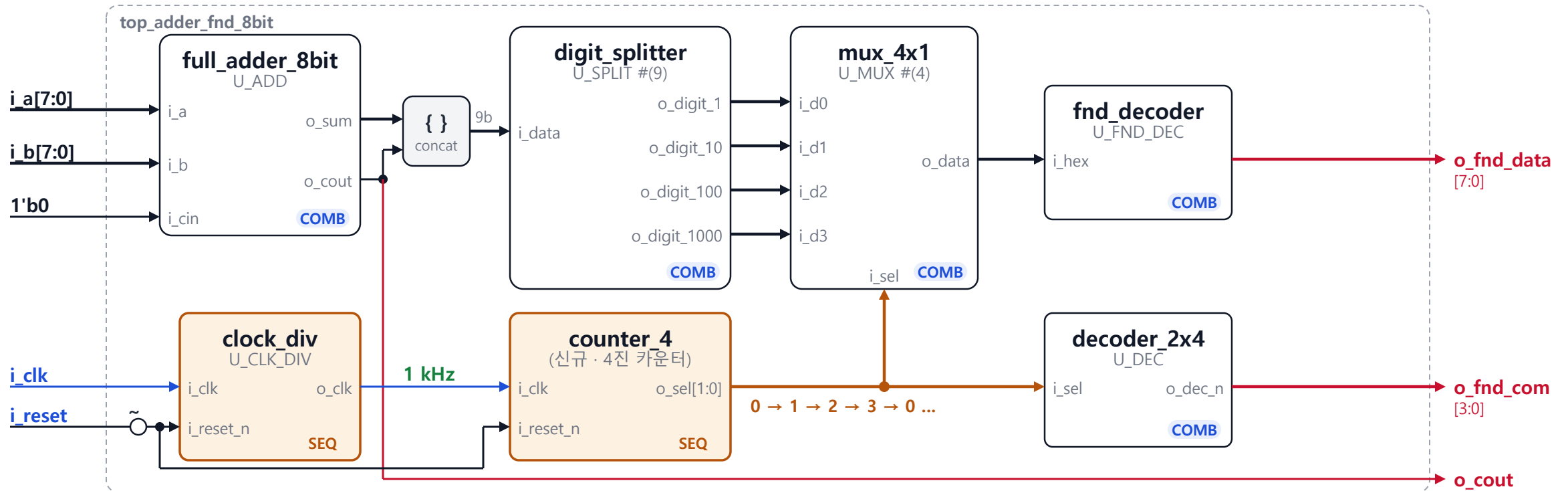
② 같은 번호를 두 곳에

번호가 MUX(어떤 숫자)와 디코더(어느 자리)에 동시에 들어가 숫자와 자리가 항상 일치

③ 문제 —

자리당 10 ns, 4자리 한 바퀴(1 frame) 40 ns.
권장(자리당 ≥ 0.25 ms)보다 25,000배 짧아 옆 자리에 잔상 발생

Clock Divider Added — Each Digit Now Stays Lit for 1 ms, No Ghosting



COMB 조합 회로 SEQ 순차 회로 잔상 문제 → 100 MHz를 1 kHz로 분주해 카운터에 공급

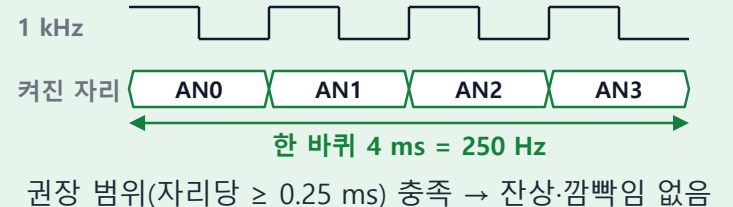
1 클럭 분주기 (clock_div)

100 MHz 클럭을 세어 100,000개마다 한 바퀴 → 1 kHz 클럭
 앞 50,000개는 1, 뒤 50,000개는 0 (Duty 50%)
 원리·카운터 폭은 다음 장

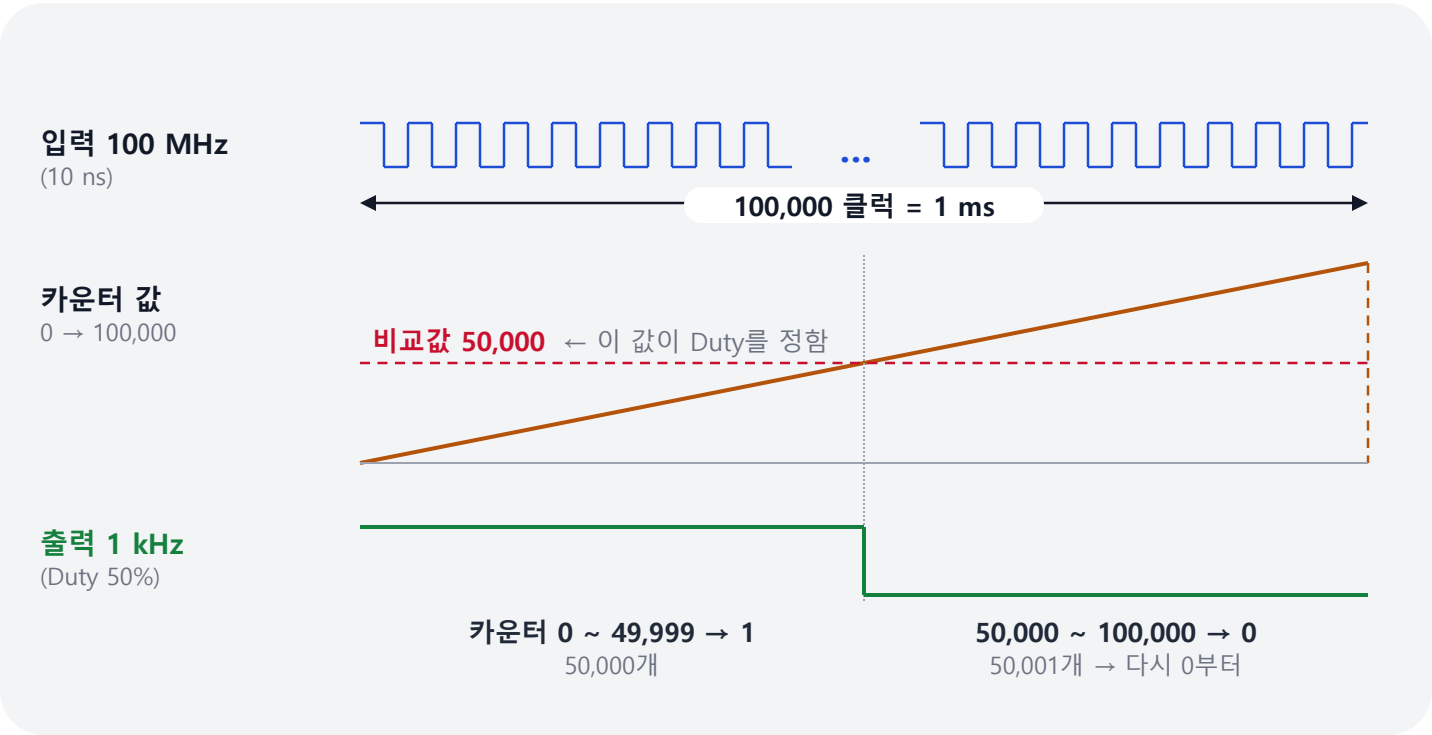
2 4진 카운터 (counter_4)

1 kHz가 올라갈 때마다 +1 → 번호가 1 ms씩 유지
 번호 0~3 = AN0~AN3, MUX·디코더에 동시에 전달

3 결과 — 자리당 1 ms



Divide 100 MHz by 100,000 — A 17-bit Counter Is the Minimum for 1 kHz



÷ 100,000

분주비

100 MHz ÷ 1 kHz = 100,000. 자리당 1 ms · 한 바퀴 4 ms로
계산이 쉬워 1 kHz 선택

17 bit

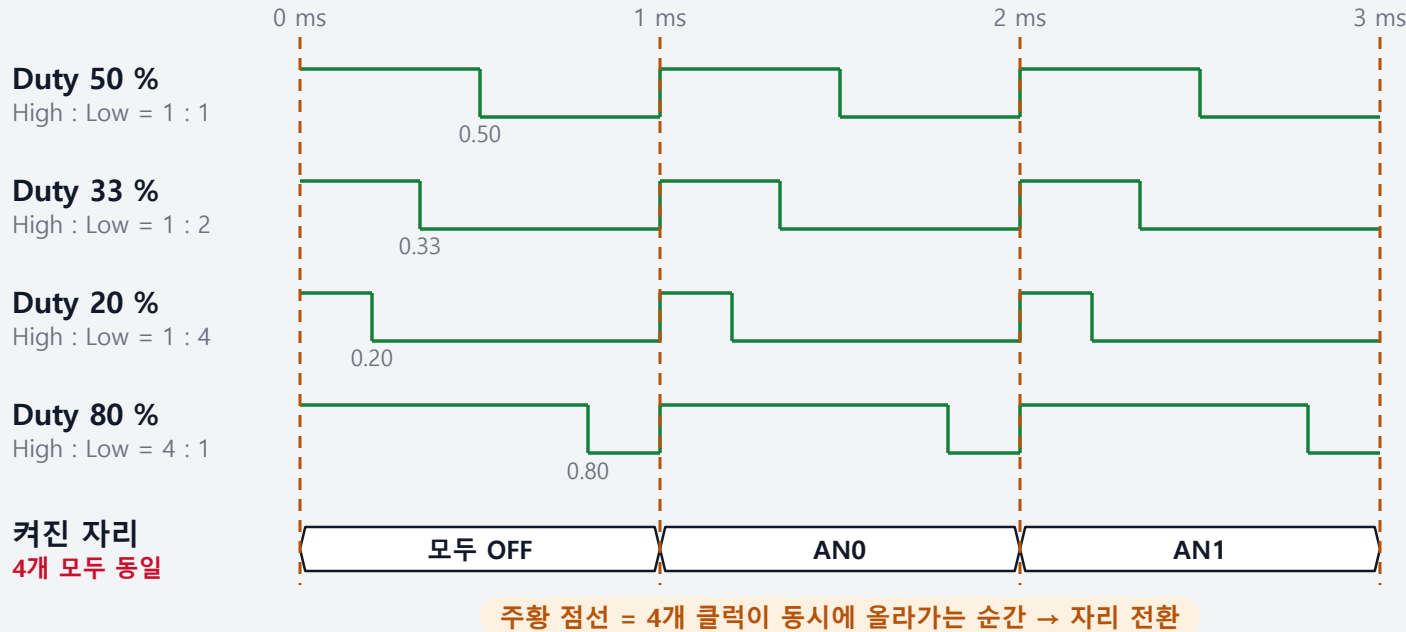
필요한 카운터 폭

$2^{16} = 65,536 < 100,000 \rightarrow$ **16bit(64k)로는 못 셈.** 최소
17bit(최대 131,071), 코드는 20bit로 여유

비교값 변경				
High : Low 비교값 Duty	1 : 1 50,000 50 %	1 : 2 33,334 33 %	1 : 4 20,000 20 %	4 : 1 80,000 80 %

Duty 1:1 · 1:2 · 1:4 · 4:1 — Identical Scan, Because All Clocks Rise Together

파형 (Vivado xsim 실측값으로 그림)



결과가 같은 이유

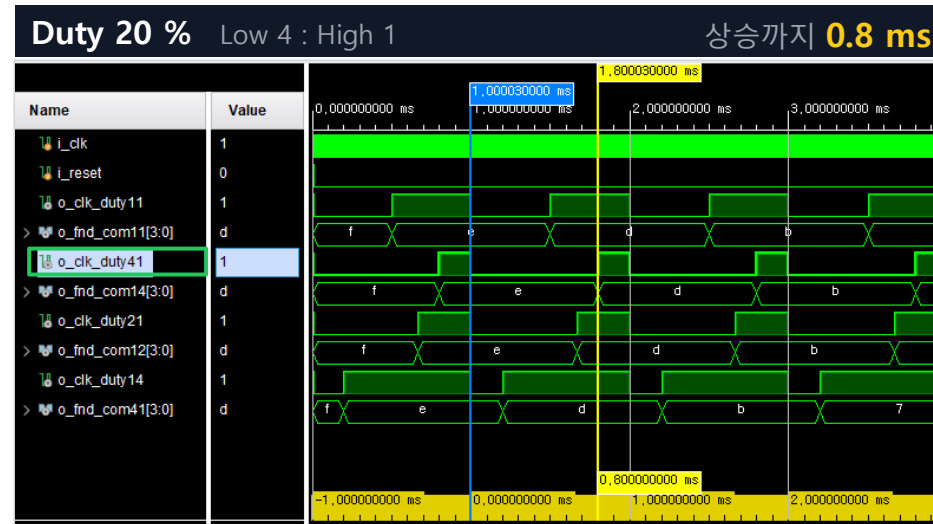
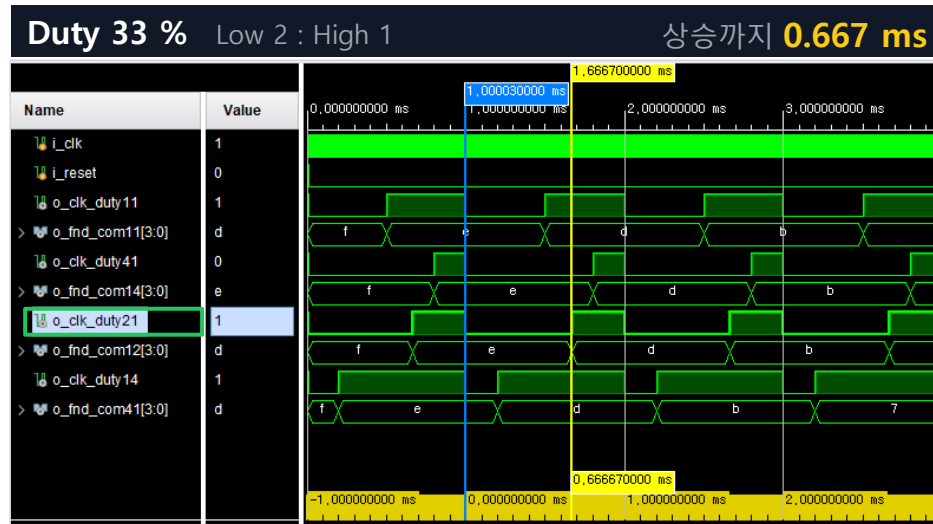
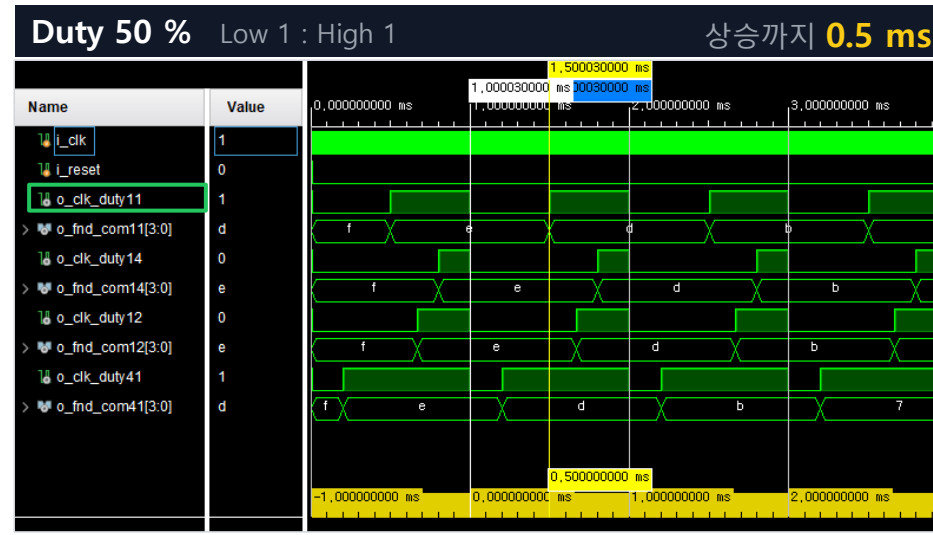
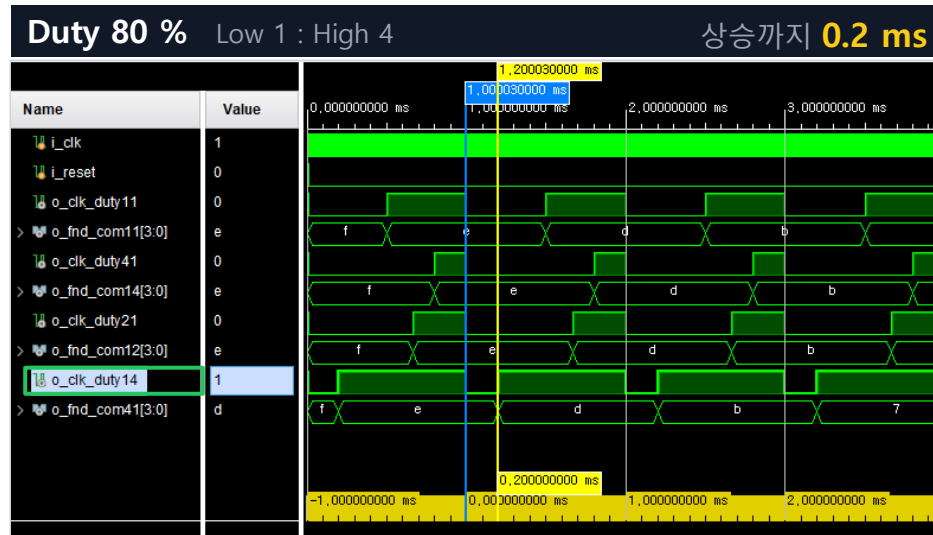
- 1 같은 출발점
4개 분주기 모두 0~100,000을 세고 같은 순간 처음으로 → 주기와 올라가는 시점 동일
- 2 바뀐 건 떨어지는 지점뿐
비교값(50,000 / 33,334 / 20,000 / 80,000)은 1 → 0으로 떨어지는 시점만 결정
- 3 FSM은 올라가는 엣지에서만 동작
자리 전환은 올라가는 엣지에서만 → 떨어지는 시점은 영향 없음

실측: 자리 전환 1.00003 · 2.00004 · 3.00005 ms — 4개 모두 같은 시각

결론 — 1이 먼저 나오는 분주기에서는 Duty를 바꿔도 화면 동작은 4개 모두 동일

- Duty 차이는 클럭 파형에만 보이고 자리 전환(Anode)에는 나타나지 않음
- 차이를 보려면 올라가는 시점 변경 필요 → 0이 먼저 나오도록 바꿔서 다시 측정 (다음 장)

Low-First Clock — The Rising Edge Moves with Duty: 0.2 / 0.5 / 0.67 / 0.8 ms



읽는 법

파란 커서 1.00003 ms —
4개 클럭이 함께 0으로
떨어지는 순간

노란 커서 — 연두 박스
클럭이 올라가는 순간

두 커서 차이 = **Low 구간** =
그 클럭의 자리 전환 시점

측정값

Duty	상승
80 %	0.2 ms
50 %	0.5 ms
33 %	0.667 ms
20 %	0.8 ms

Low 구간 =

$$(1 - \text{Duty}) \times 1 \text{ ms}$$

상승 엣지가 Low 구간만큼
밀려 자리 전환 시점이
모두 다름

Conclusion and Discussion

1 설계 결과

3단계 개선으로 4자리 동시 표시
1자리 표시 → 고스팅 → 클럭 분주로 해결
보드에서 최대 510까지 정상 출력



하지만 “보드에서 맞게 보인다”
≠ “검증 끝”

2 기존 TB의 문제

최종 숫자 값만 비교
화면이 어떻게 켜지는지는 보지 않음
버그 ② 천의 자리 꺼짐
버그 ④ 순서 건너뛴 → PASS로 통과
40 ns 고정 채점
스캔 속도가 바뀌면 쓸 수 없음

3 AI와 함께 한 비교 검증

- ① Claude와 기존 TB를 분석해 허점 6가지 도출
- ② 버그 5개를 심은 회로로 기존 TB와 새 TB를 같은 조건에서 비교 (Vivado)
- ③ 새 TB: 131,072개 전수 PASS, 버그 5개 모두 검출

고찰 스위치를 바꾸는 순간 값이 흔들림 → 비동기 입력이 원인, 입력 동기화(2-FF)·디바운스 방법 학습 후 적용 예정

Next Step 분주 클럭 → Clock Enable 구조로 전환, 새 TB를 이후 과제의 기본 검증 틀로 사용

핵심 교훈 버그를 일부러 심어 TB가 잡아내는지까지 확인해야 정확한 검증 가능